Neural Network

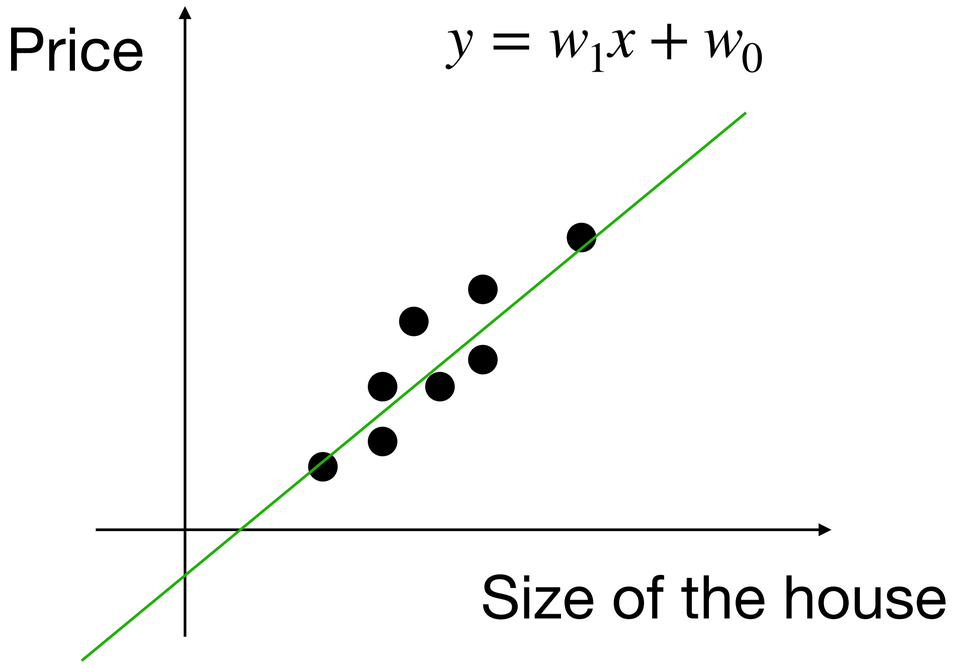
Outline of Today

1. Analysis of Boosting

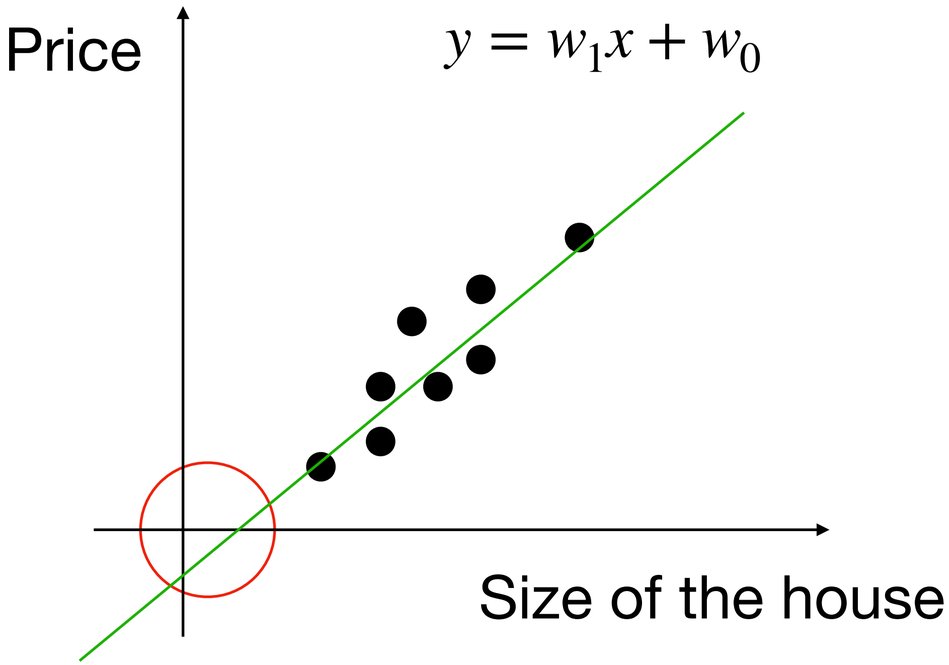
2. Multilayer feedforward Neural Network

3. Training a neural network

Linear Regression Revisit

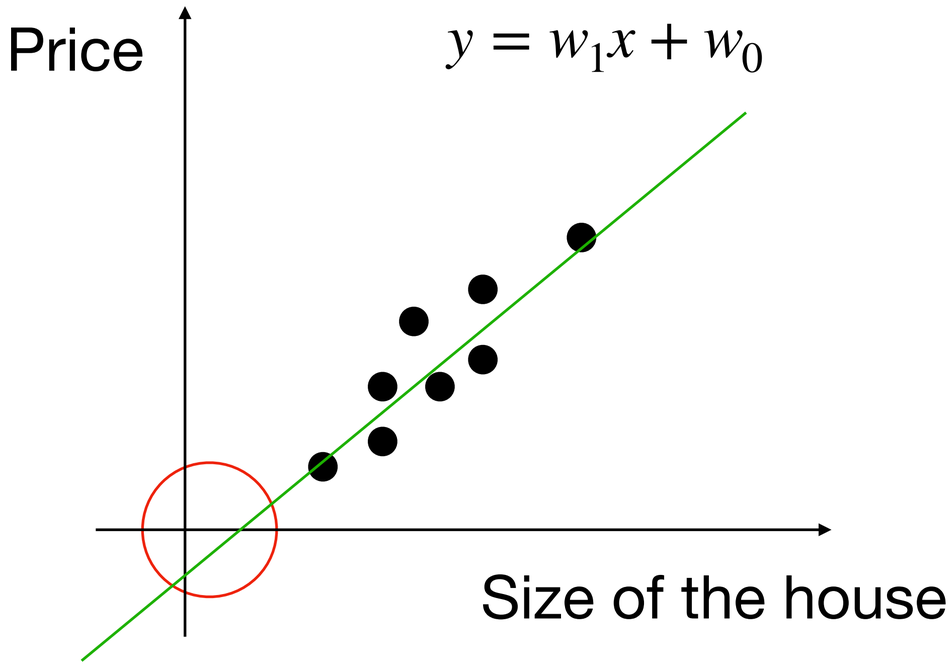


Linear Regression Revisit



Negative part does not make too much sense

Linear Regression Revisit

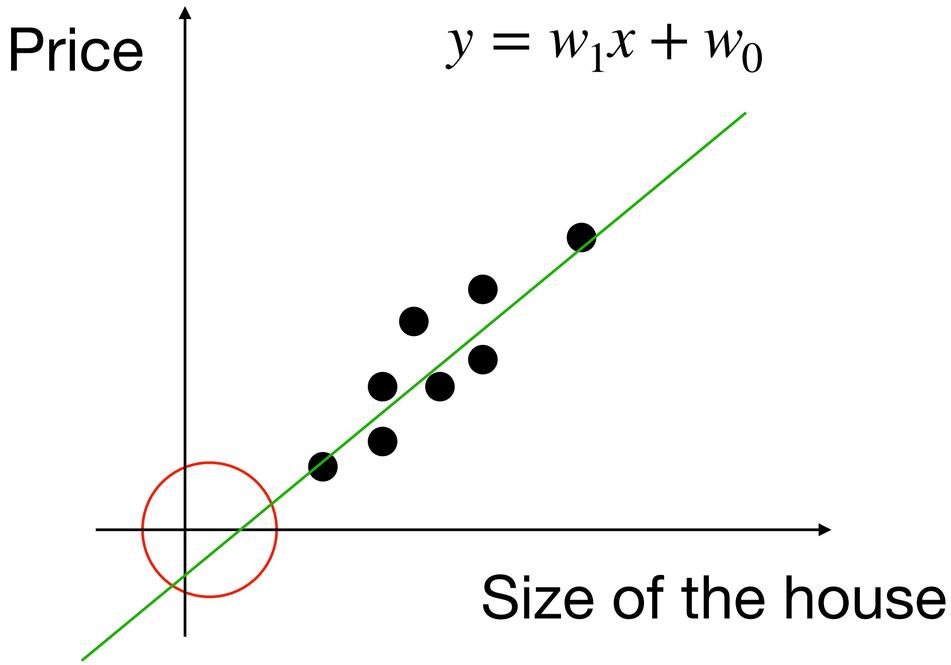


Negative part does not make too much sense

We can fix this with a simple nonlinear function

$$y = \max\{w_1 x + w_0, 0\}$$

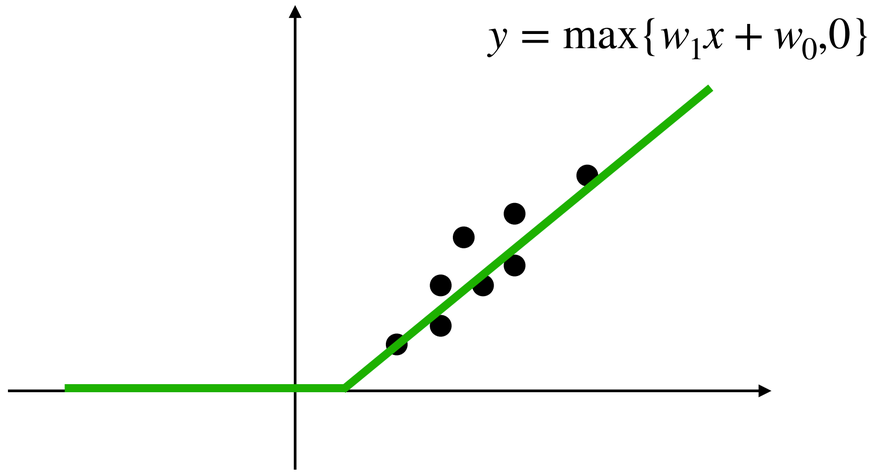
Linear Regression Revisit



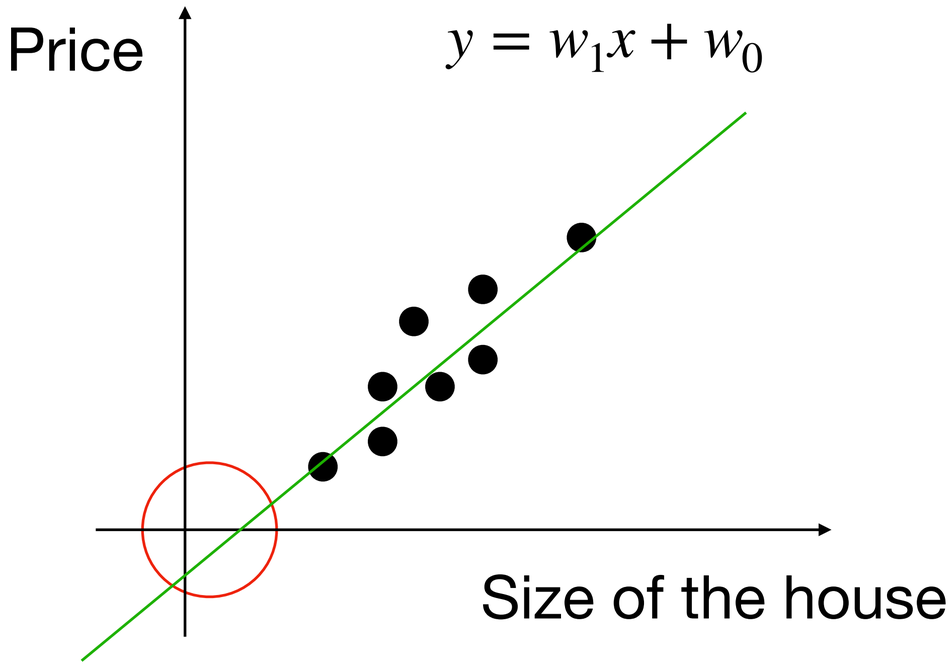
Negative part does not make too much sense

We can fix this with a simple nonlinear function

$$y = \max\{w_1 x + w_0, 0\}$$



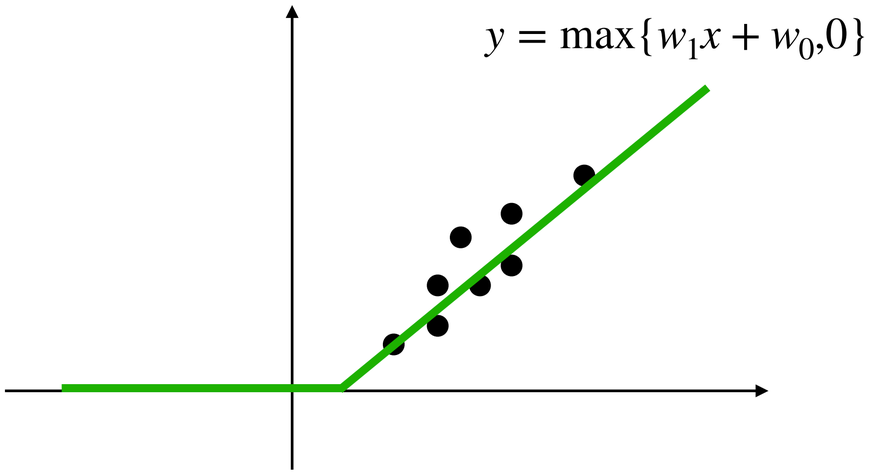
Linear Regression Revisit



Negative part does not make too much sense

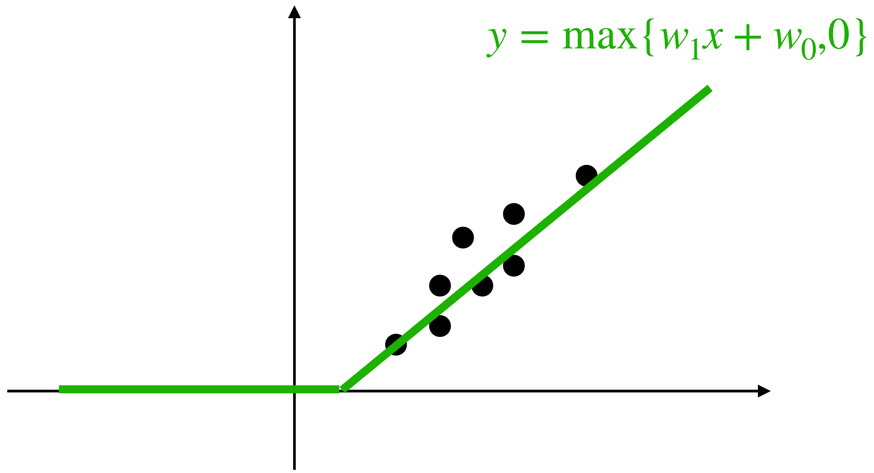
We can fix this with a simple nonlinear function

$$y = \max\{w_1 x + w_0, 0\}$$

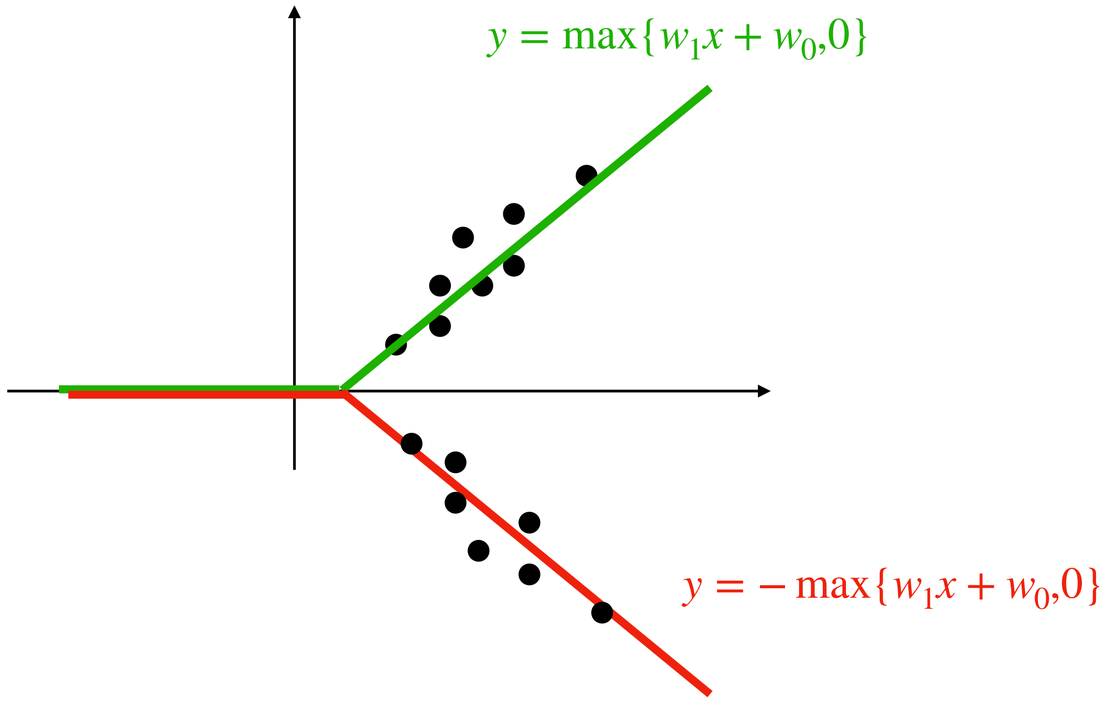


rectified linear unit (ReLU)

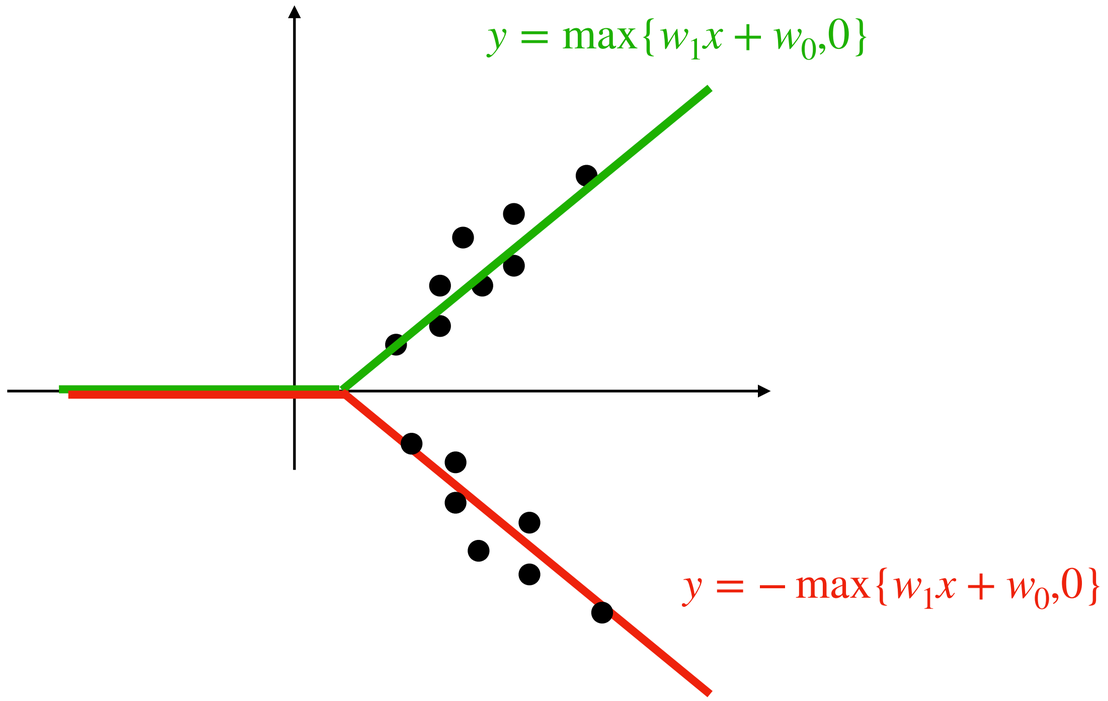
A single neuron network



A single neuron network

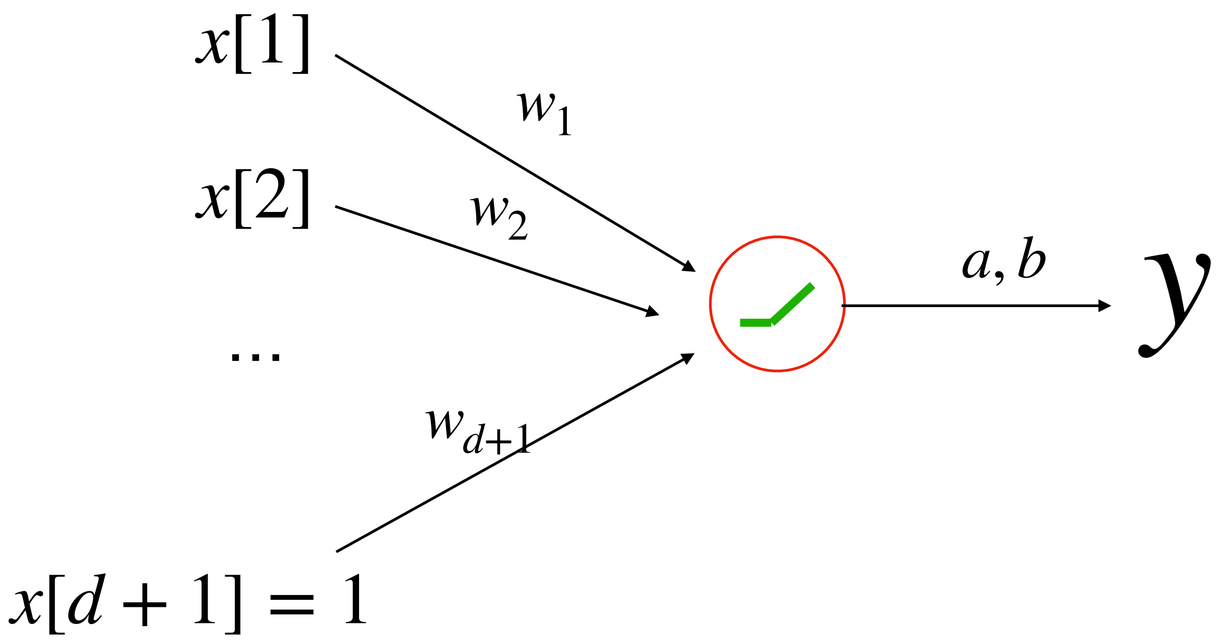


A single neuron network

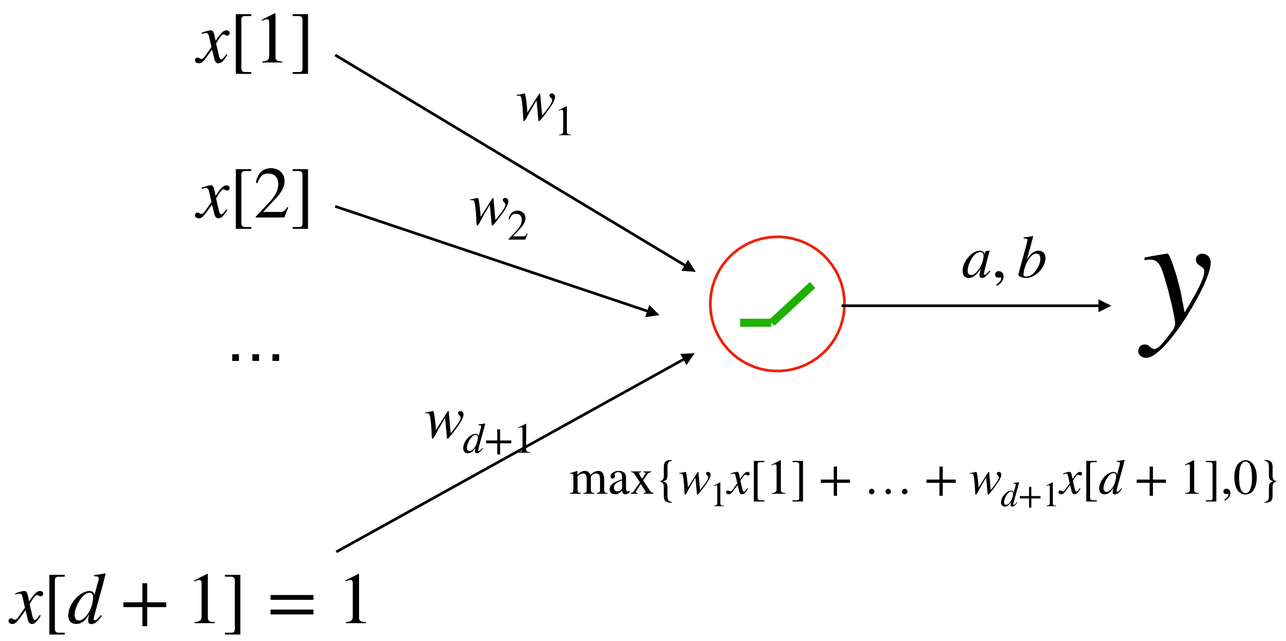


$$y = a \max\{w_1 x + w_0, 0\} + b$$

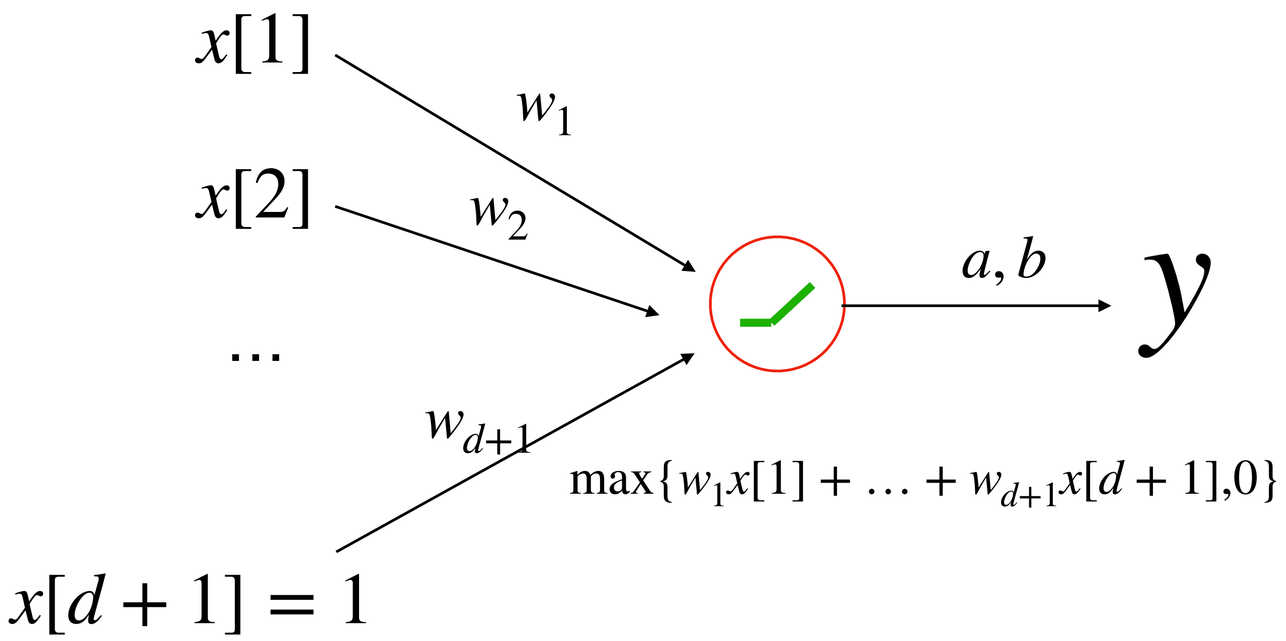
A single neuron network



A single neuron network

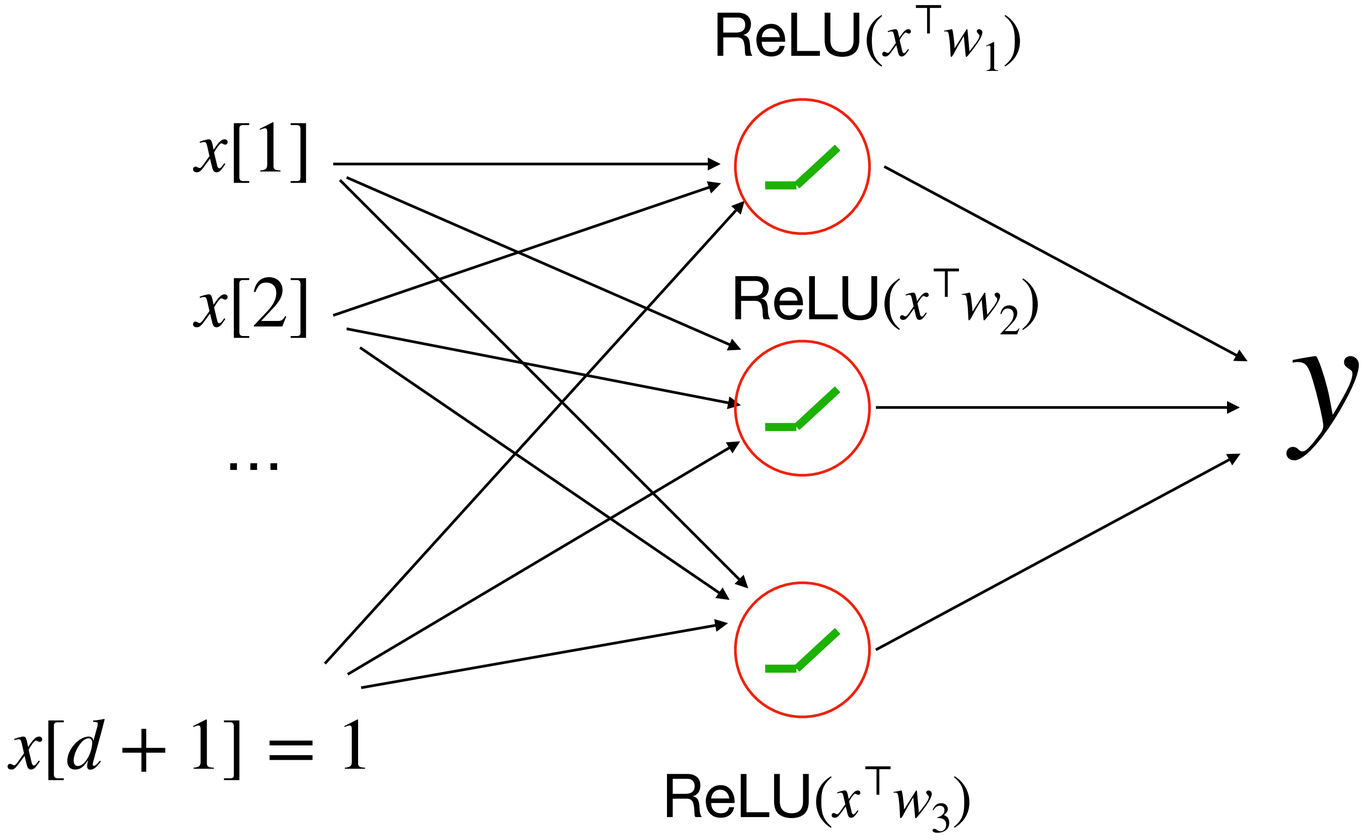


A single neuron network



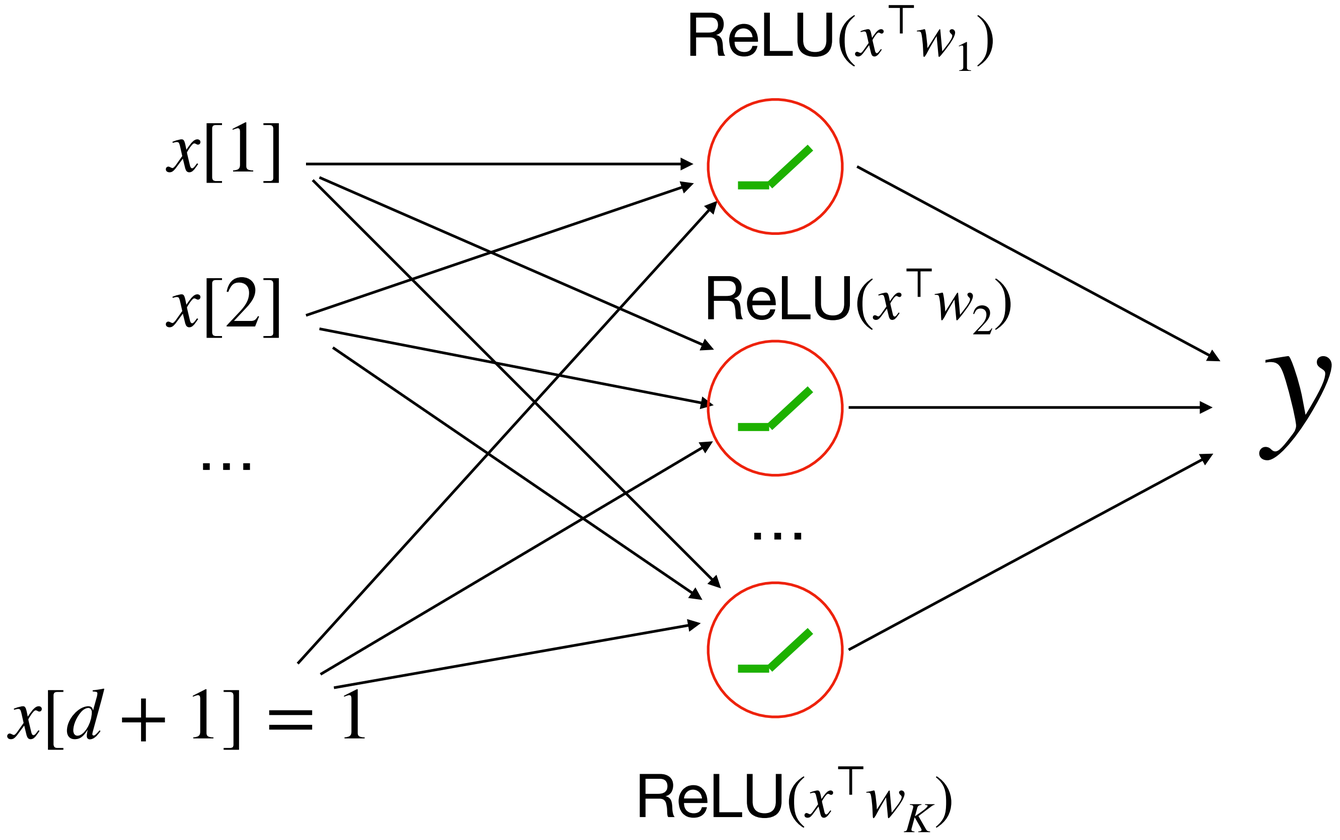
$$y = a\mathrm{ReLU}(w^\top x) + b$$

Let us stack multiple neurons together

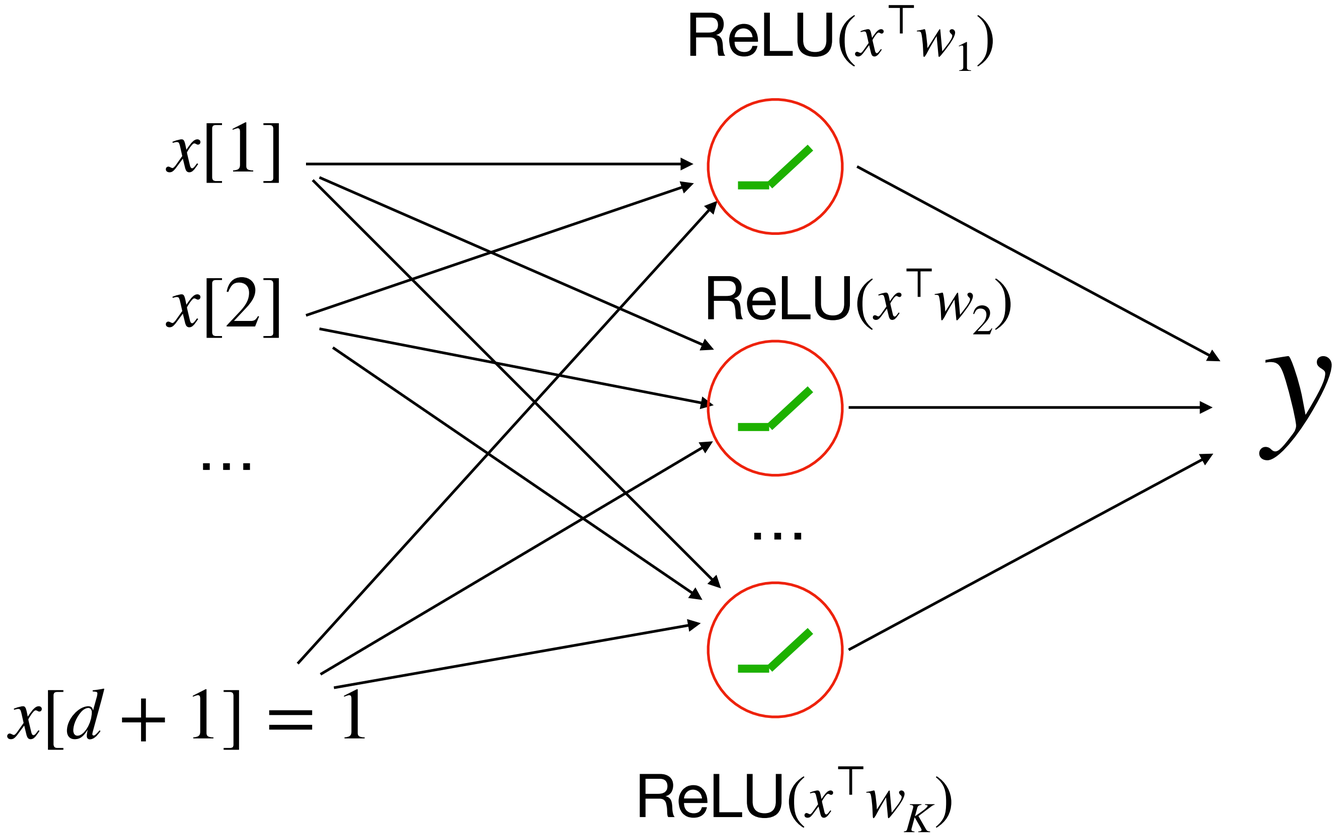


$$y = \sum_{i=1}^{3} a_i \mathrm{ReLU}(x^\top w_i) + b$$

Let us stack multiple neurons together



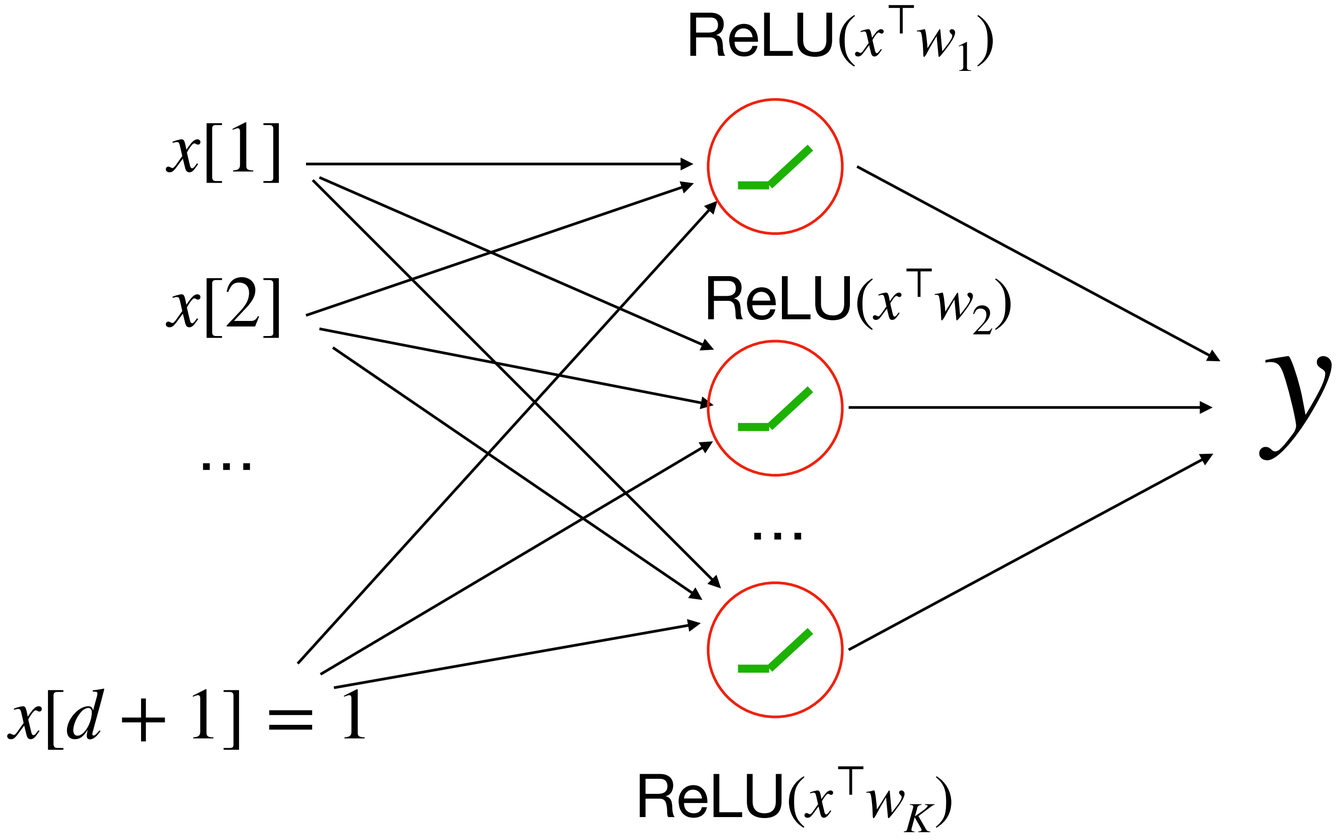
Let us stack multiple neurons together



Vectorized form:

Define $W = \begin{bmatrix} (w_1)^\top \\ \cdots \\ (w_K)^\top \end{bmatrix} \in \mathbb{R}^{K \times d}$

Let us stack multiple neurons together

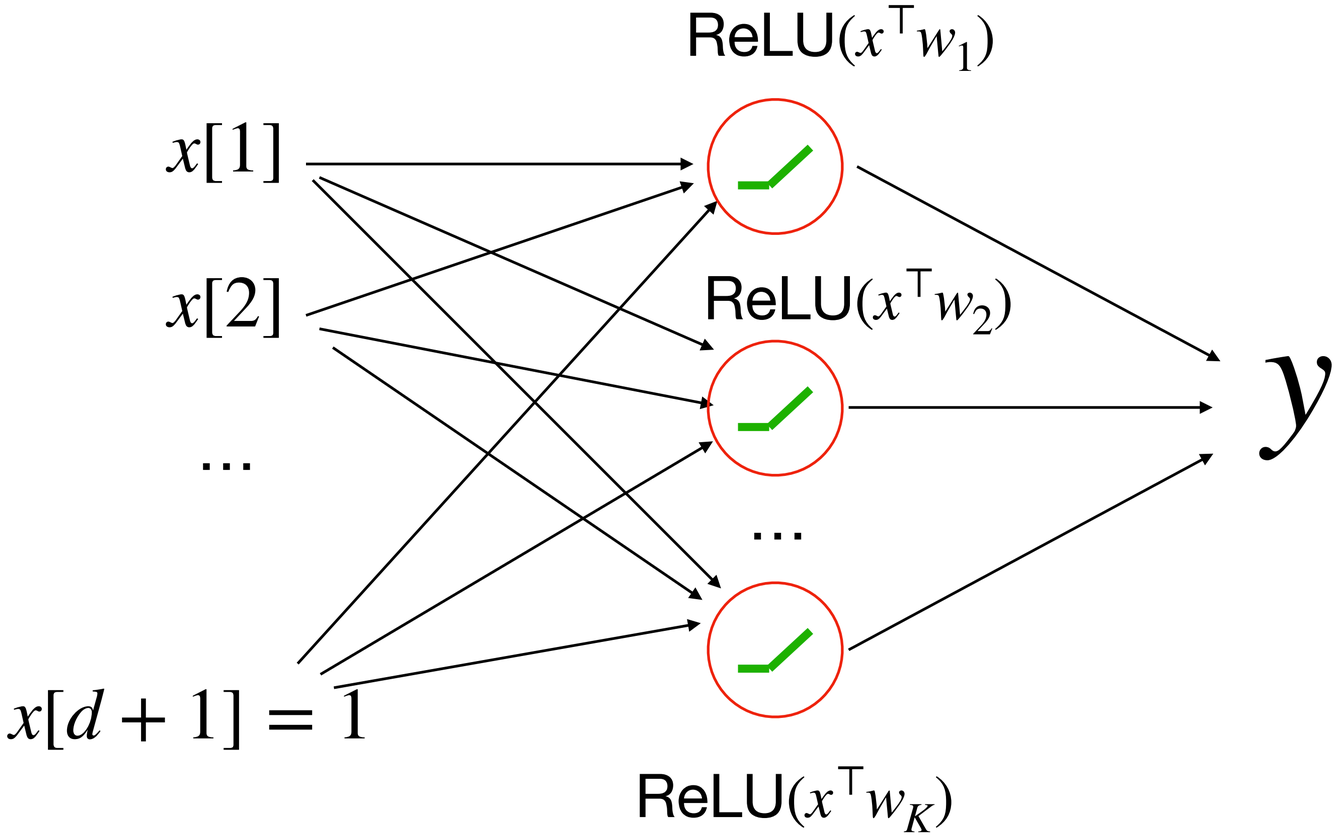


Vectorized form:

Define $W = \begin{bmatrix} (w_1)^\top \\ \cdots \\ (w_K)^\top \end{bmatrix} \in \mathbb{R}^{K \times d}$

$$\alpha = [a_1, \ldots, a_K]^\top$$

Let us stack multiple neurons together



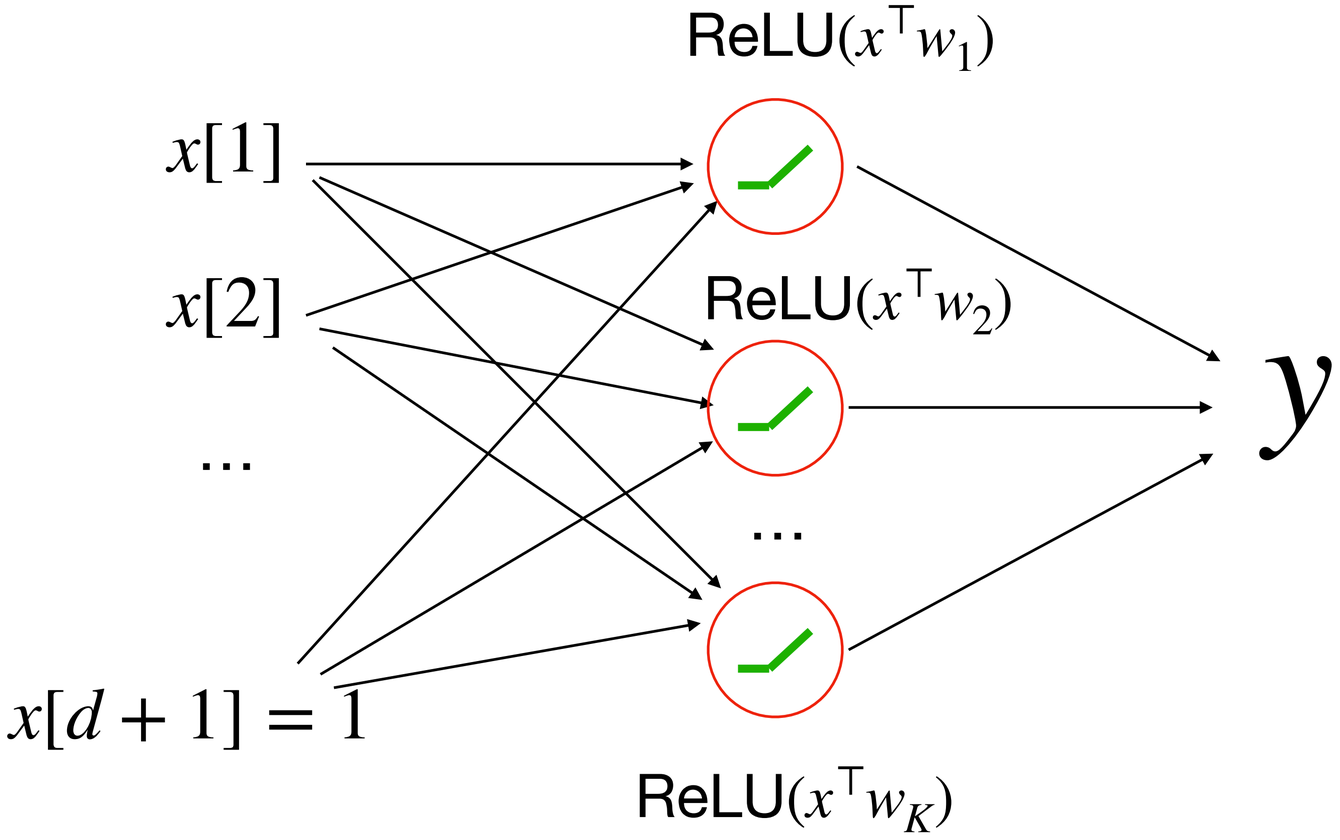
Vectorized form:

Define $W = \begin{bmatrix} (w_1)^\top \\ \cdots \\ (w_K)^\top \end{bmatrix} \in \mathbb{R}^{K \times d}$

$$\alpha = [a_1, \ldots, a_K]^\top$$

$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$

Let us stack multiple neurons together



Vectorized form:

Define $W = \begin{bmatrix} (w_1)^\top \\ \cdots \\ (w_K)^\top \end{bmatrix} \in \mathbb{R}^{K \times d}$

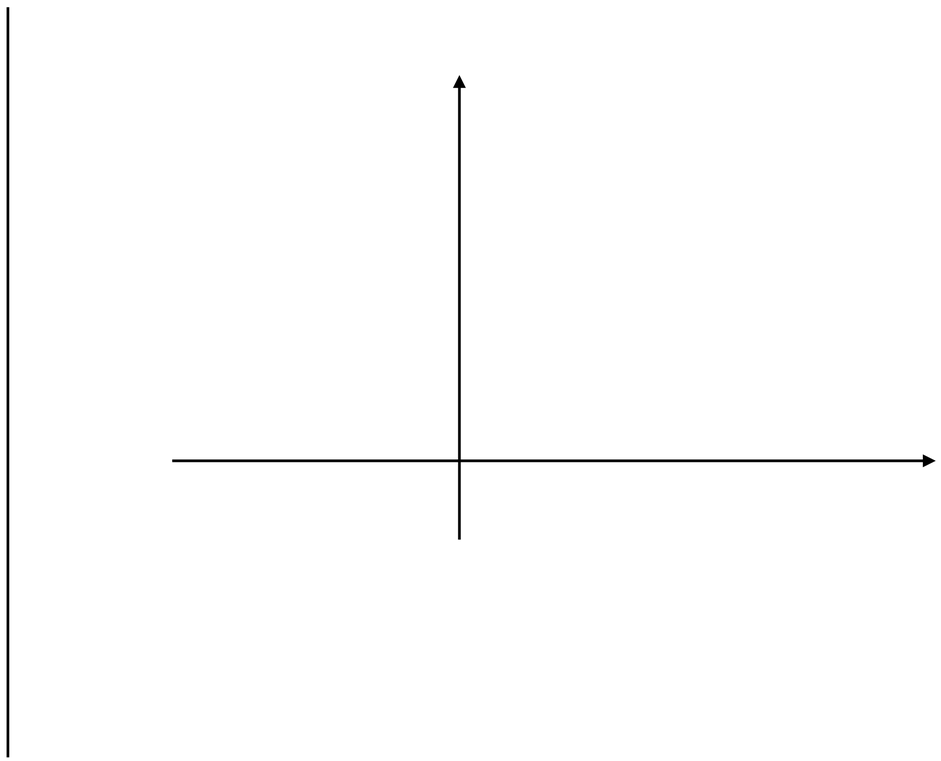
$$\alpha = [a_1, \ldots, a_K]^\top$$

$$y = \alpha^\top \big(\text{ReLU}(Wx)\big) + b$$

Learnable feature $\phi(x)$

What does a neural network approximate

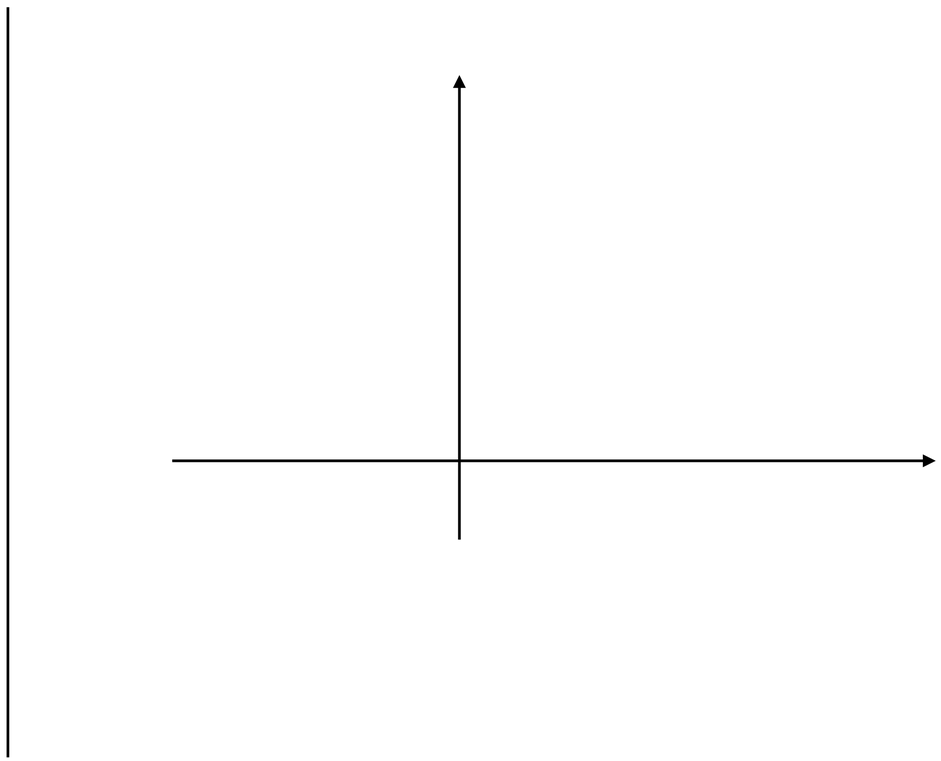
$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$



What does a neural network approximate

$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$

It's a pieces wise linear functions

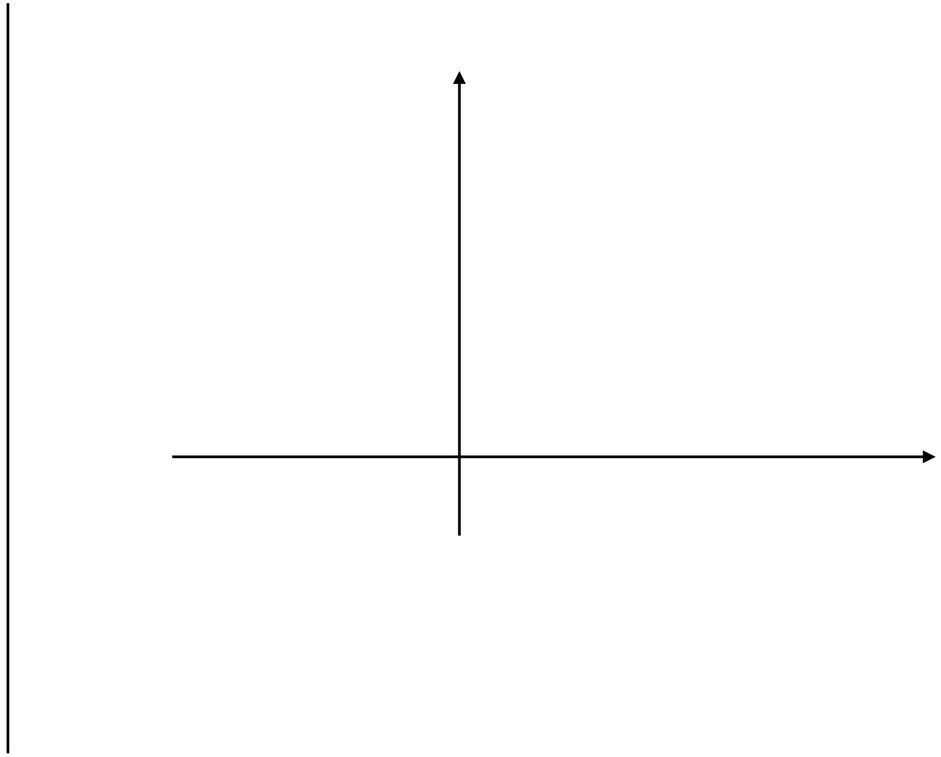


What does a neural network approximate

$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$

It's a pieces wise linear functions

Consider $d = 1$ case (and assume $b = 0$):



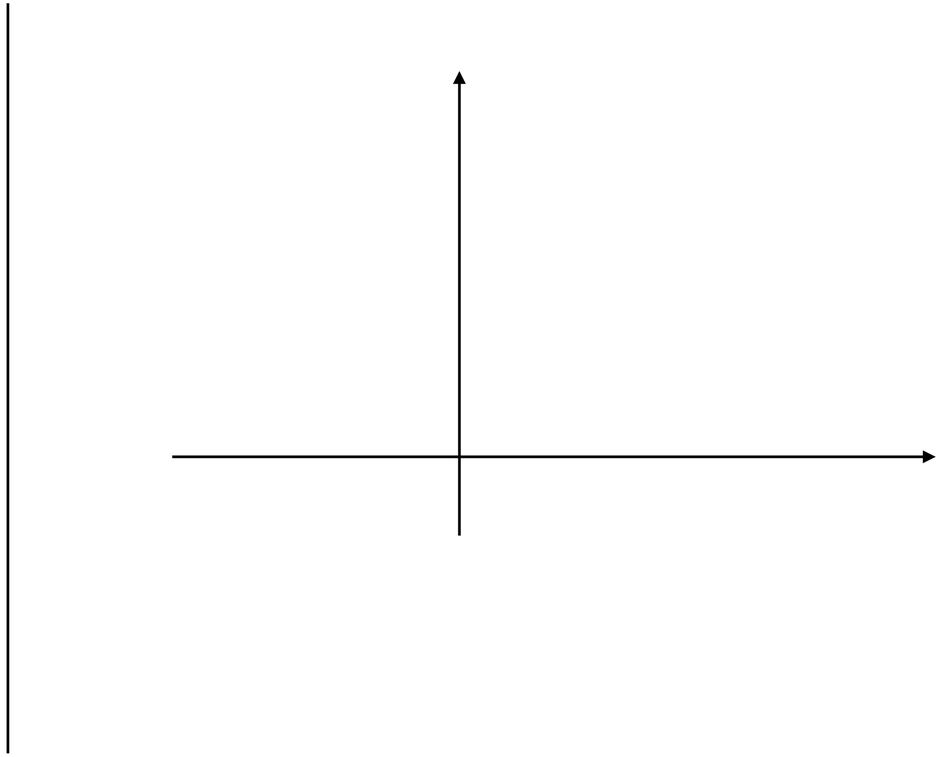
What does a neural network approximate

$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$

It's a pieces wise linear functions

Consider $d = 1$ case (and assume $b = 0$):

$$K = 1 : y = a_1 \max\{w_1 x + c_1, 0\}$$



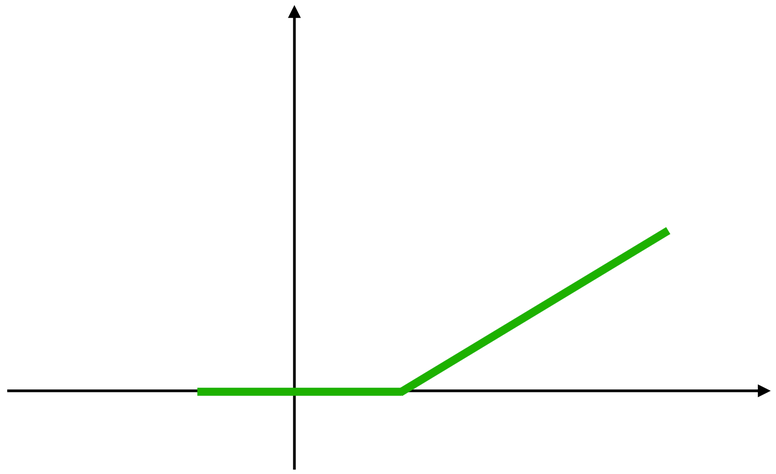
What does a neural network approximate

$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$

It's a pieces wise linear functions

Consider $d = 1$ case (and assume $b = 0$):

$$K = 1 : y = a_1 \max\{w_1 x + c_1, 0\}$$



What does a neural network approximate

$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$

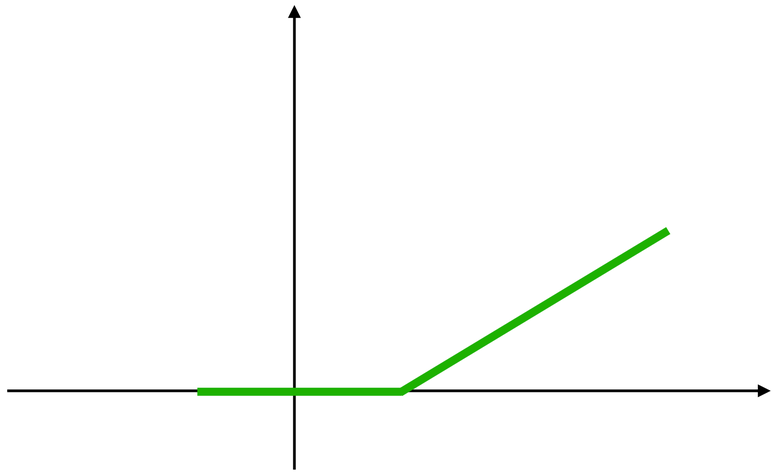
It's a pieces wise linear functions

Consider $d = 1$ case (and assume $b = 0$):

$$K = 1 : y = a_1 \max\{w_1 x + c_1, 0\}$$

$$K = 2 : y = a_1 \max\{w_1 x + c_1, 0\}$$

$$+ a_2 \max\{w_2 x + c_2, 0\}$$



What does a neural network approximate

$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$

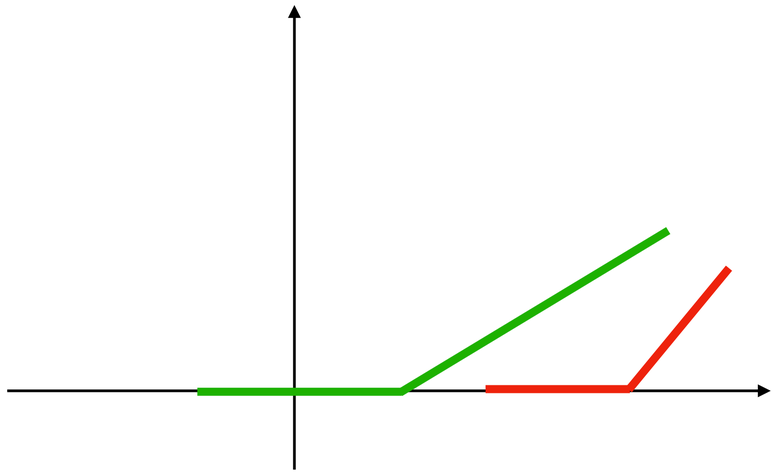
It's a pieces wise linear functions

Consider $d = 1$ case (and assume $b = 0$):

$$K = 1 : y = a_1 \max\{w_1 x + c_1, 0\}$$

$$K = 2 : y = a_1 \max\{w_1 x + c_1, 0\}$$

$$+ a_2 \max\{w_2 x + c_2, 0\}$$



What does a neural network approximate

$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$

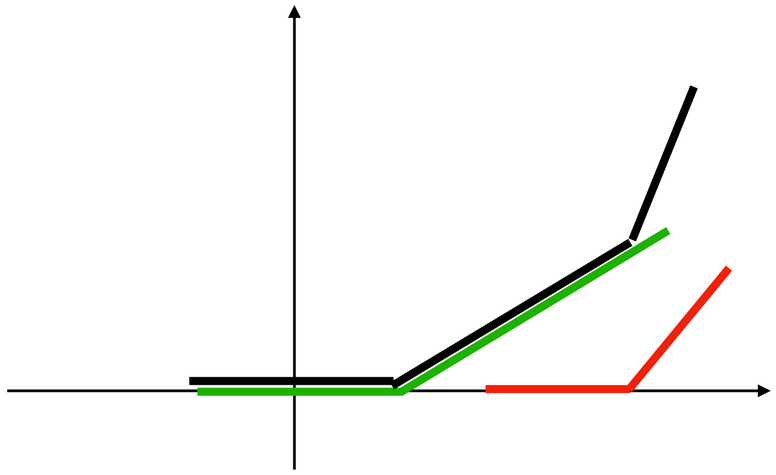
It's a pieces wise linear functions

Consider $d = 1$ case (and assume $b = 0$):

$$K = 1 : y = a_1 \max\{w_1 x + c_1, 0\}$$

$$K = 2 : y = a_1 \max\{w_1 x + c_1, 0\}$$

$$+ a_2 \max\{w_2 x + c_2, 0\}$$



What does a neural network approximate

$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$

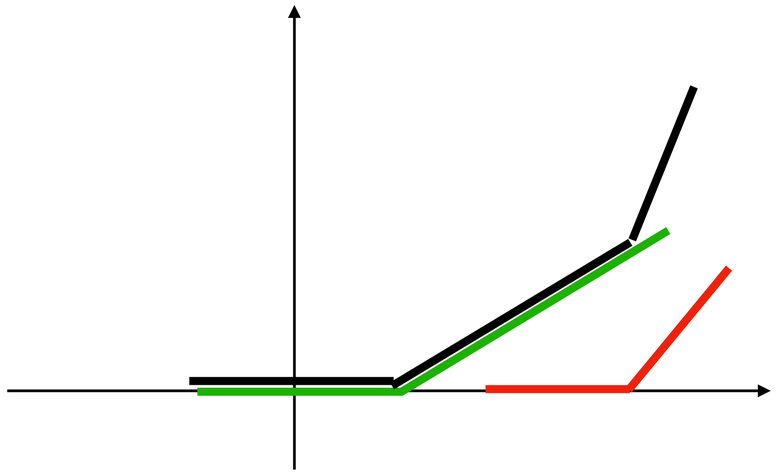
It's a pieces wise linear functions

Consider $d = 1$ case (and assume $b = 0$):

$$K = 1 : y = a_1 \max\{w_1 x + c_1, 0\}$$

$$K = 2 : y = a_1 \max\{w_1 x + c_1, 0\}$$
$$+ a_2 \max\{w_2 x + c_2, 0\}$$

$$K = 3 : y = a_1 \max\{w_1 x + c_1, 0\}$$
$$+ a_2 \max\{w_2 x + c_2, 0\}$$
$$+ a_3 \max\{w_3 x + c_3, 0\}$$



What does a neural network approximate

$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$

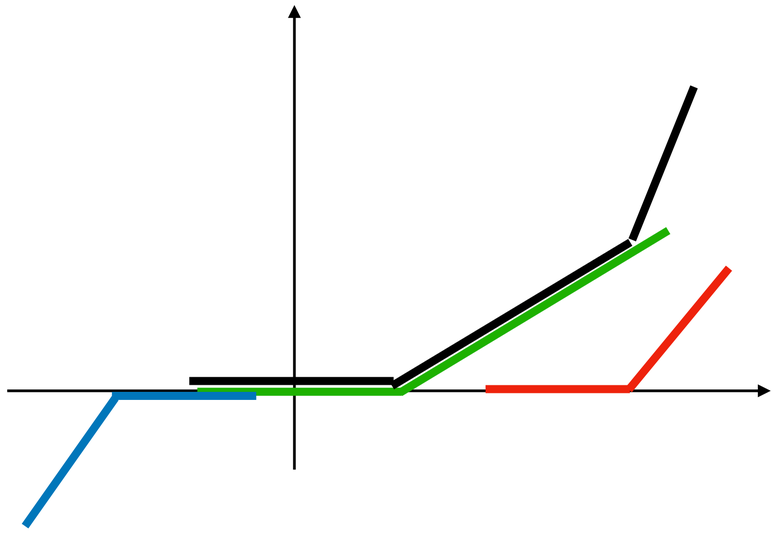
It's a pieces wise linear functions

Consider $d = 1$ case (and assume $b = 0$):

$$K = 1 : y = a_1 \max\{w_1 x + c_1, 0\}$$

$$K = 2 : y = a_1 \max\{w_1 x + c_1, 0\}$$
$$+ a_2 \max\{w_2 x + c_2, 0\}$$

$$K = 3 : y = a_1 \max\{w_1 x + c_1, 0\}$$
$$+ a_2 \max\{w_2 x + c_2, 0\}$$
$$+ a_3 \max\{w_3 x + c_3, 0\}$$



What does a neural network approximate

$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$

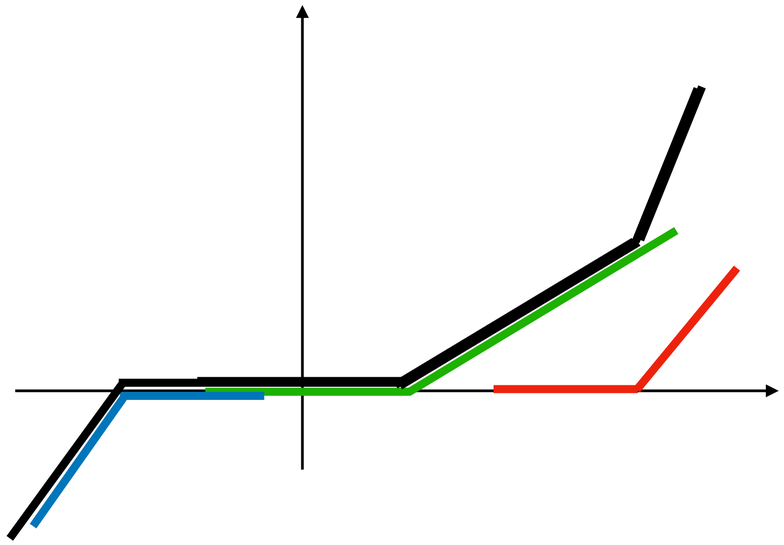
It's a pieces wise linear functions

Consider $d = 1$ case (and assume $b = 0$):

$$K = 1 : y = a_1 \max\{w_1 x + c_1, 0\}$$

$$K = 2 : y = a_1 \max\{w_1 x + c_1, 0\}$$
$$+ a_2 \max\{w_2 x + c_2, 0\}$$

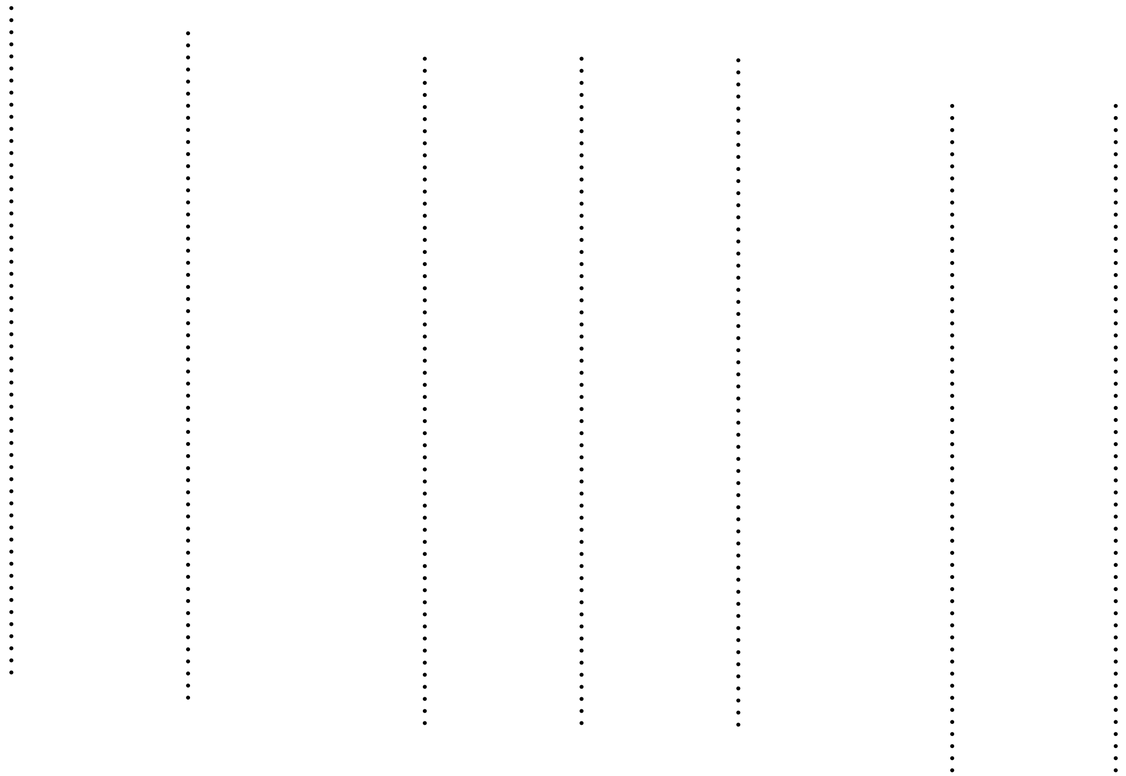
$$K = 3 : y = a_1 \max\{w_1 x + c_1, 0\}$$
$$+ a_2 \max\{w_2 x + c_2, 0\}$$
$$+ a_3 \max\{w_3 x + c_3, 0\}$$



What does a neural network approximate

$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$

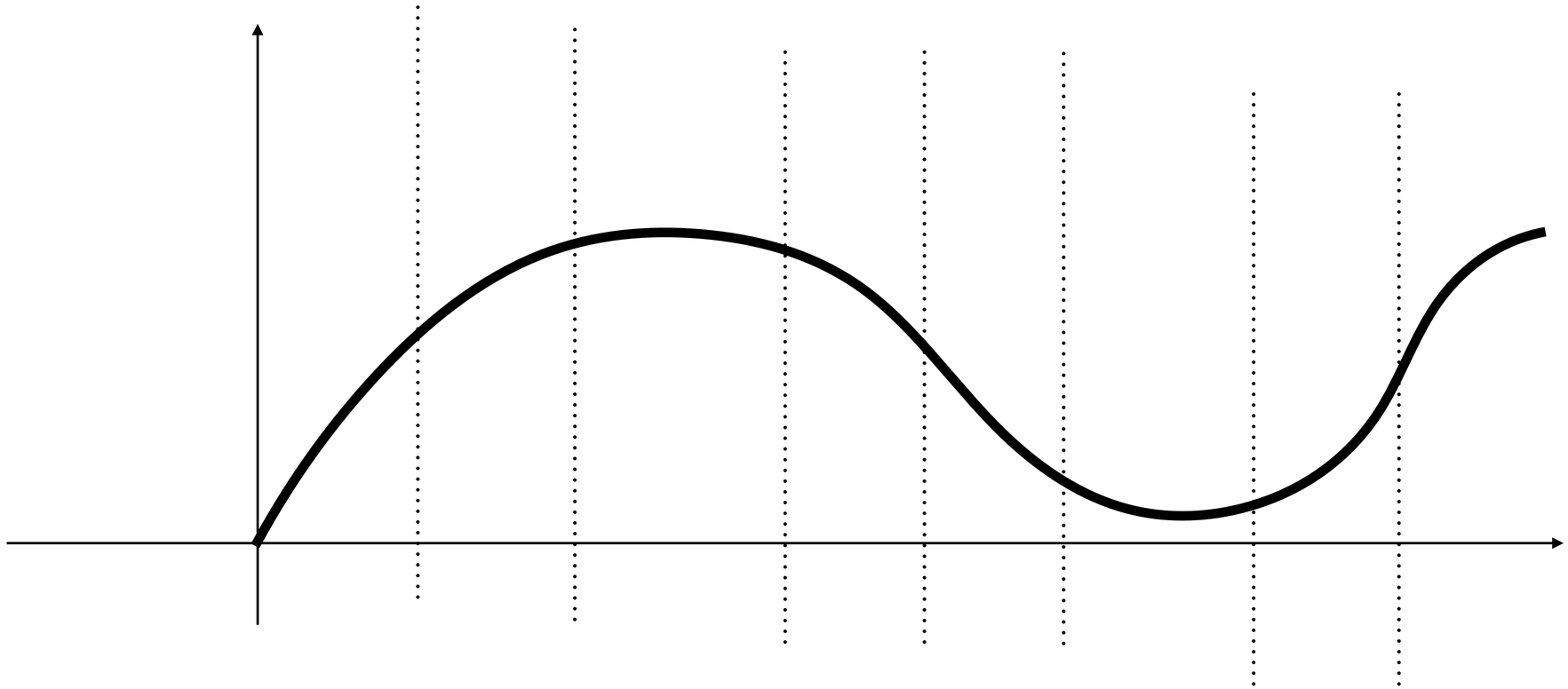
Claim: a wide enough one layer NN can approximate any smooth functions



What does a neural network approximate

$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$

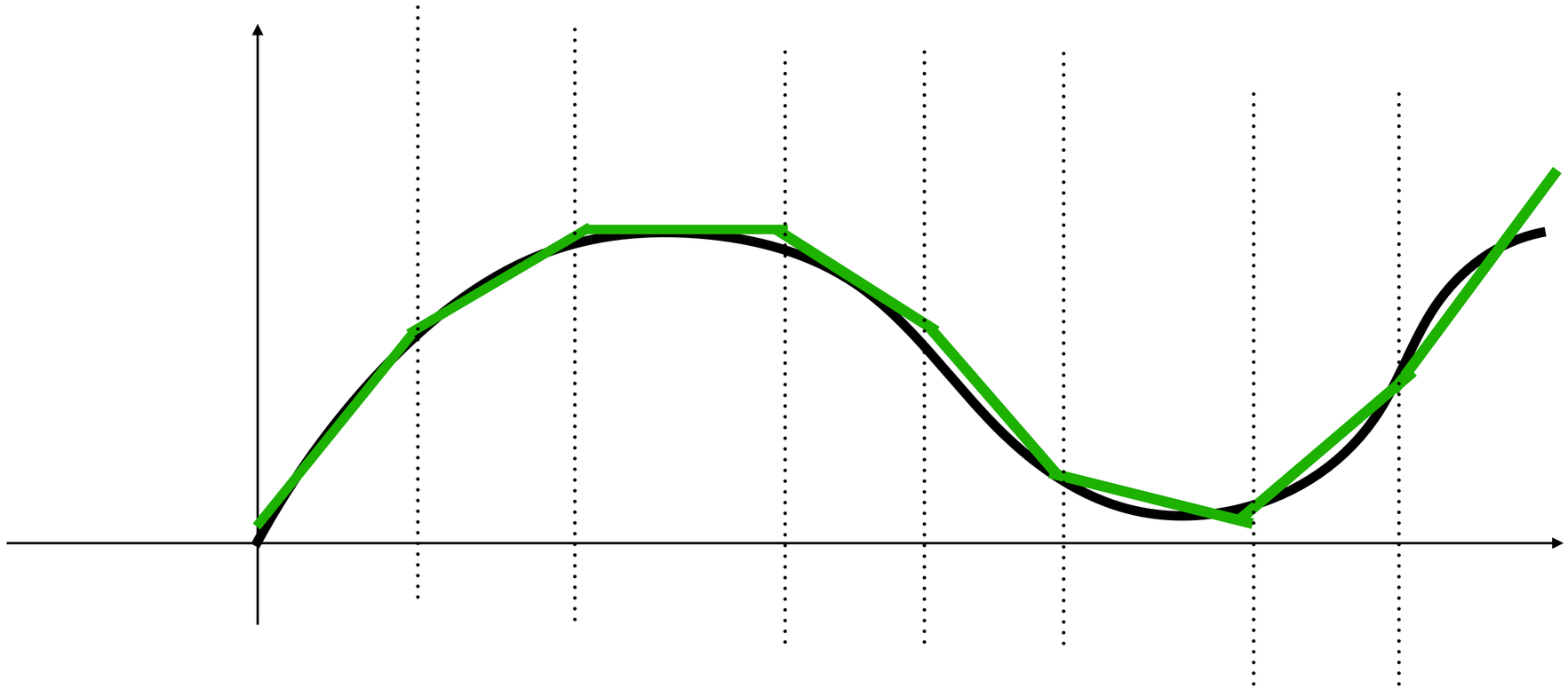
Claim: a wide enough one layer NN can approximate any smooth functions



What does a neural network approximate

$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$

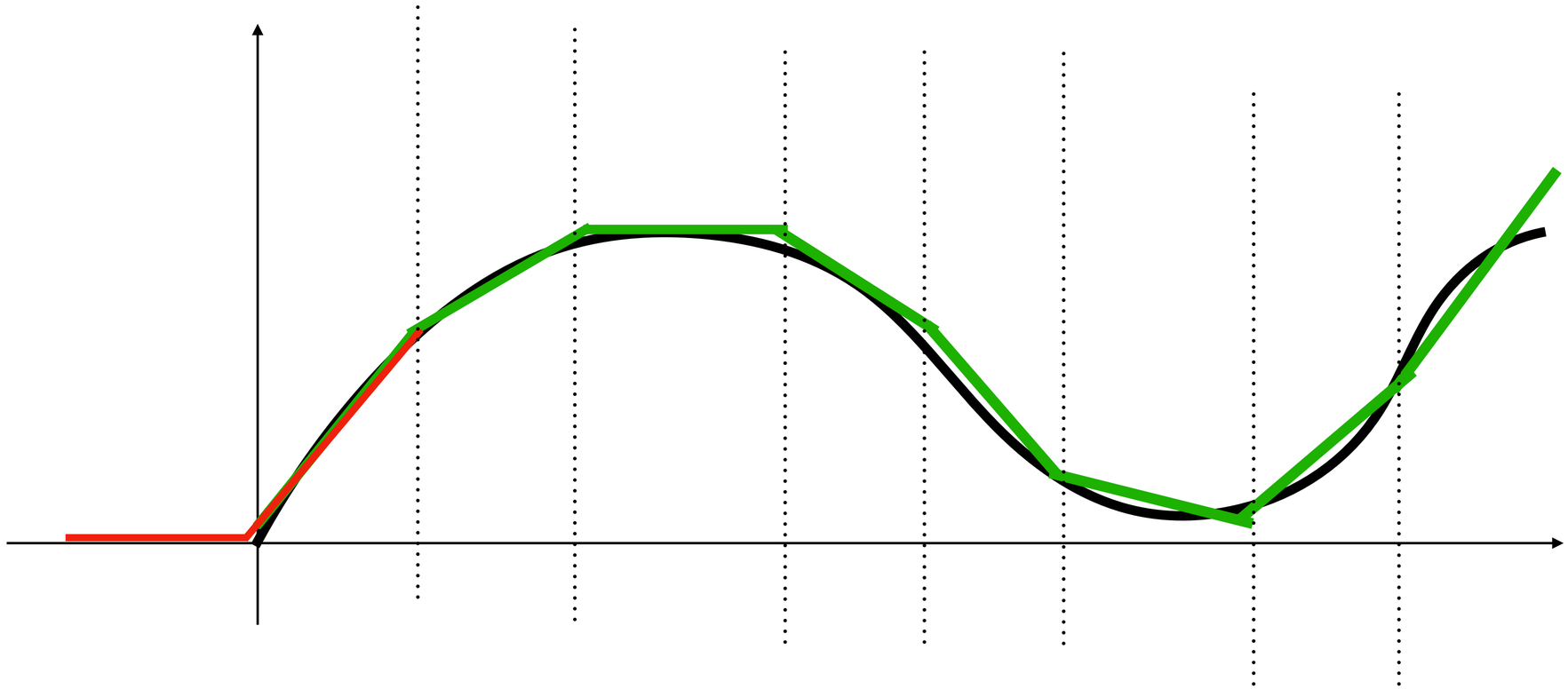
Claim: a wide enough one layer NN can approximate any smooth functions



What does a neural network approximate

$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$

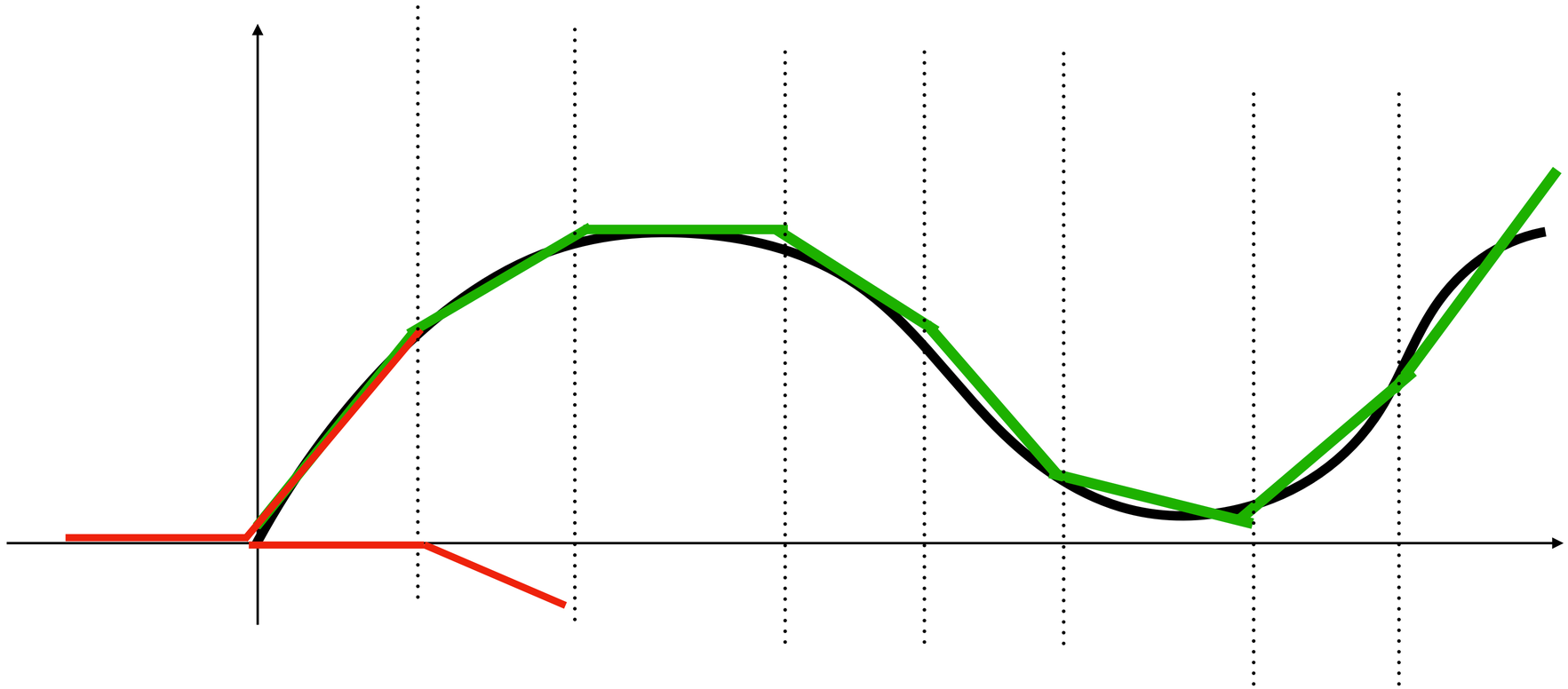
Claim: a wide enough one layer NN can approximate any smooth functions



What does a neural network approximate

$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$

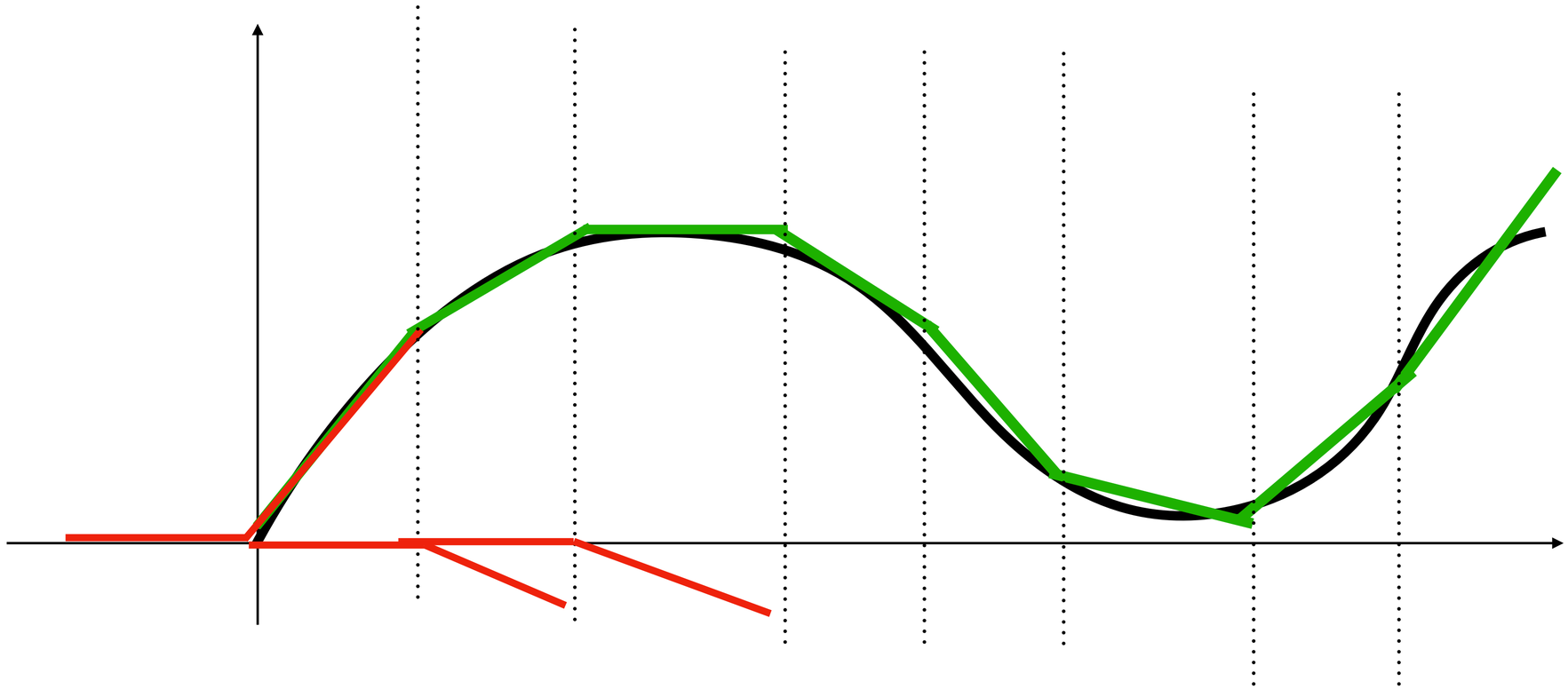
Claim: a wide enough one layer NN can approximate any smooth functions



What does a neural network approximate

$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$

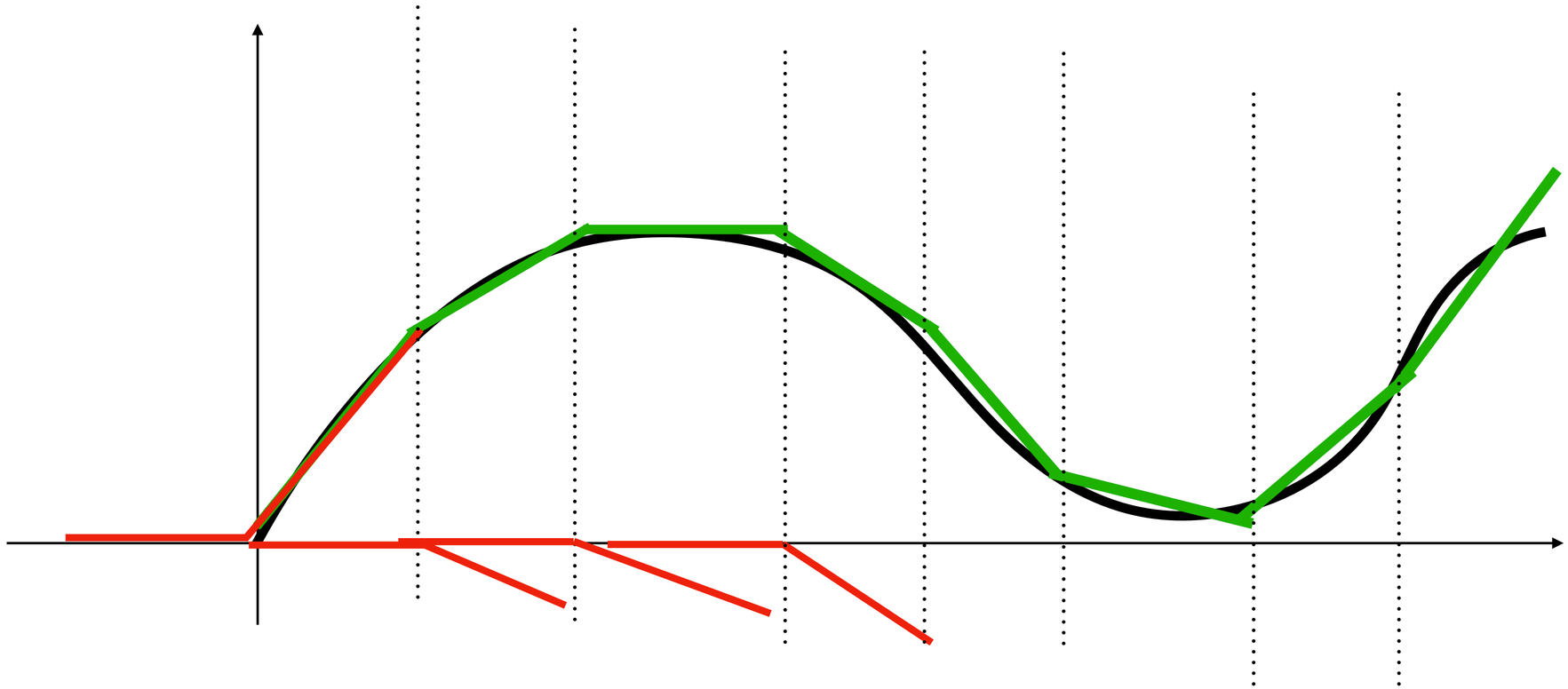
Claim: a wide enough one layer NN can approximate any smooth functions



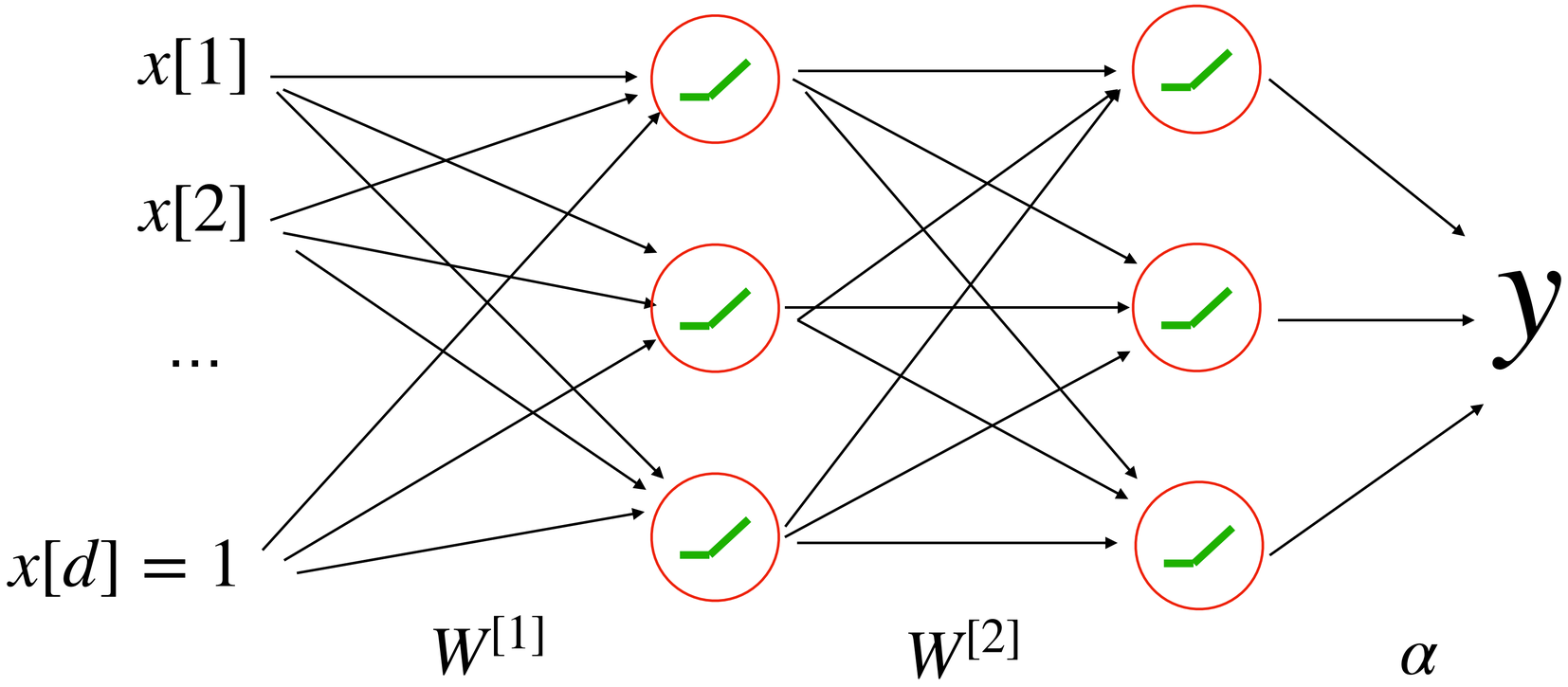
What does a neural network approximate

$$y = \alpha^\top \left(\text{ReLU}(Wx)\right) + b$$

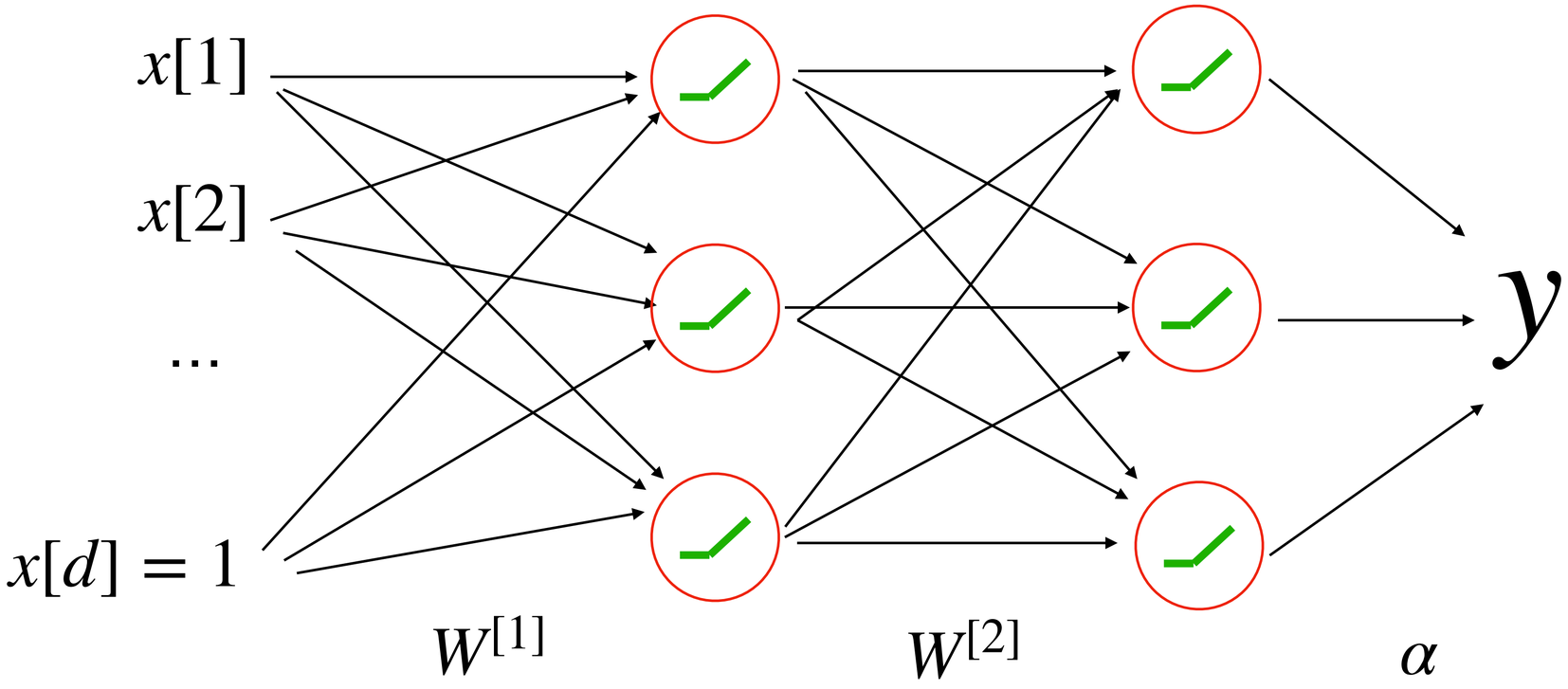
Claim: a wide enough one layer NN can approximate any smooth functions



A multi-layer fully connected neural network

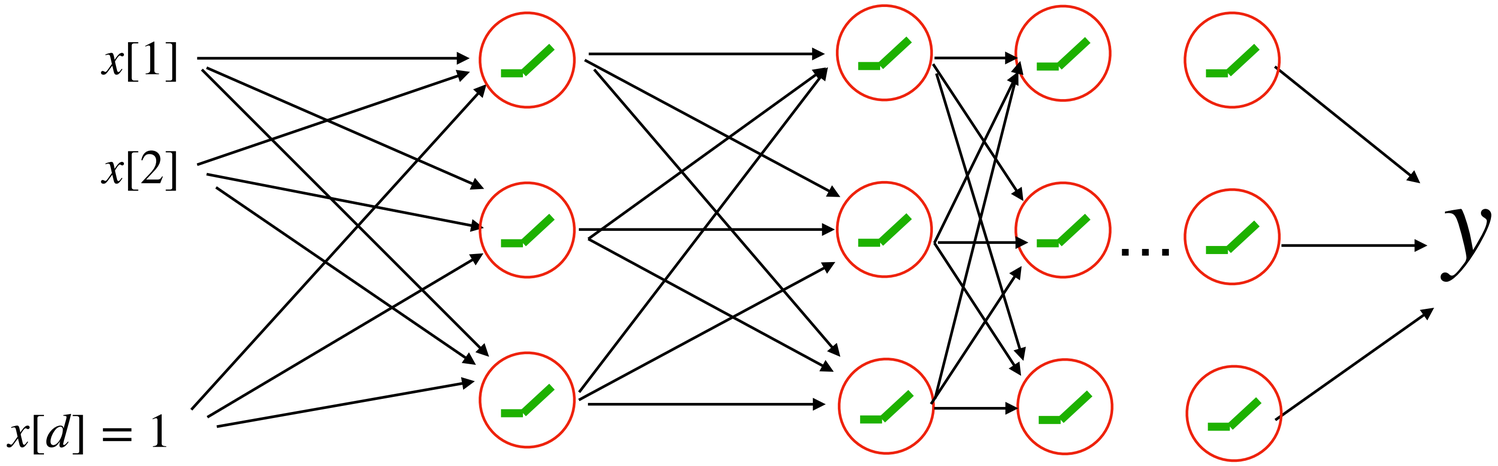


A multi-layer fully connected neural network



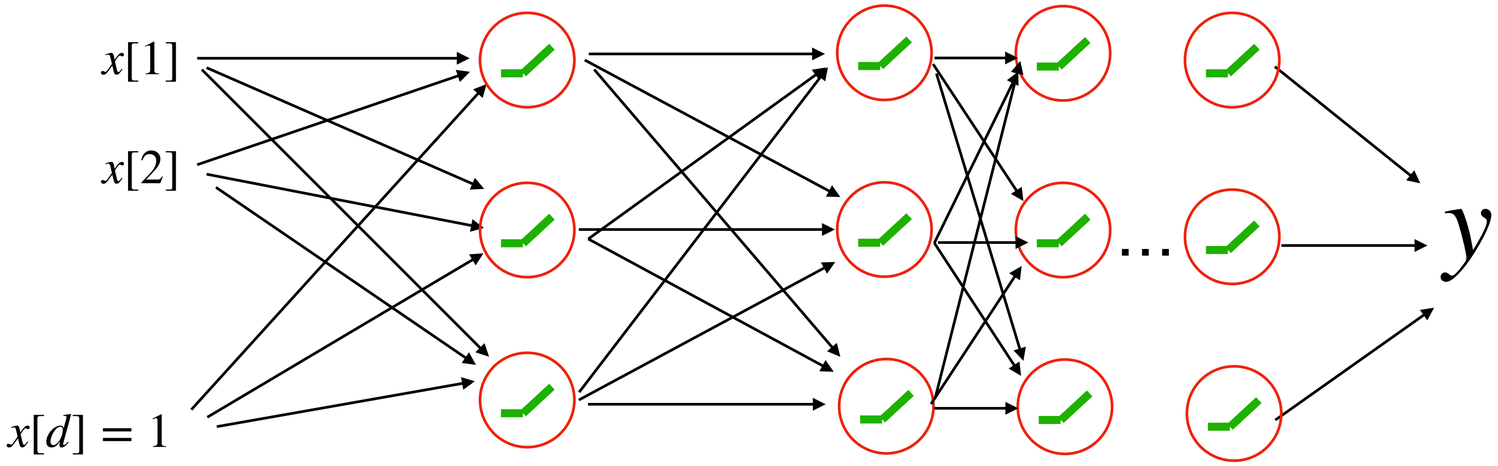
$$y = \alpha^{\top}\text{ReLU}\left(W^{[2]}\text{ReLU}\left(W^{[1]}x\right)\right) + b$$

A multi-layer fully connected neural network

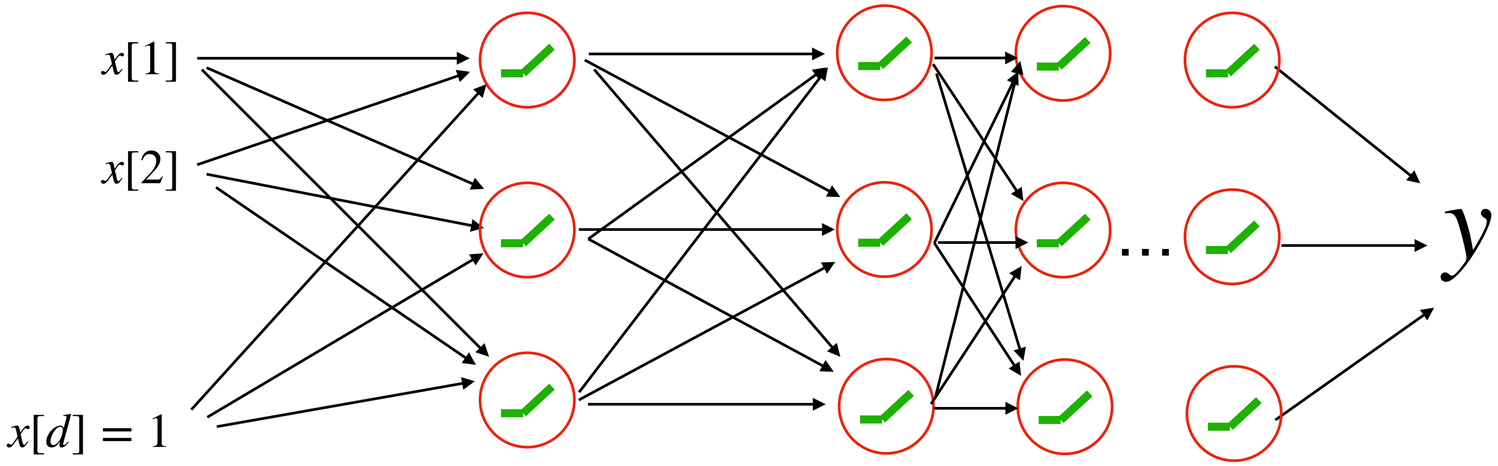


A multi-layer fully connected neural network

Define it by a forward pass:



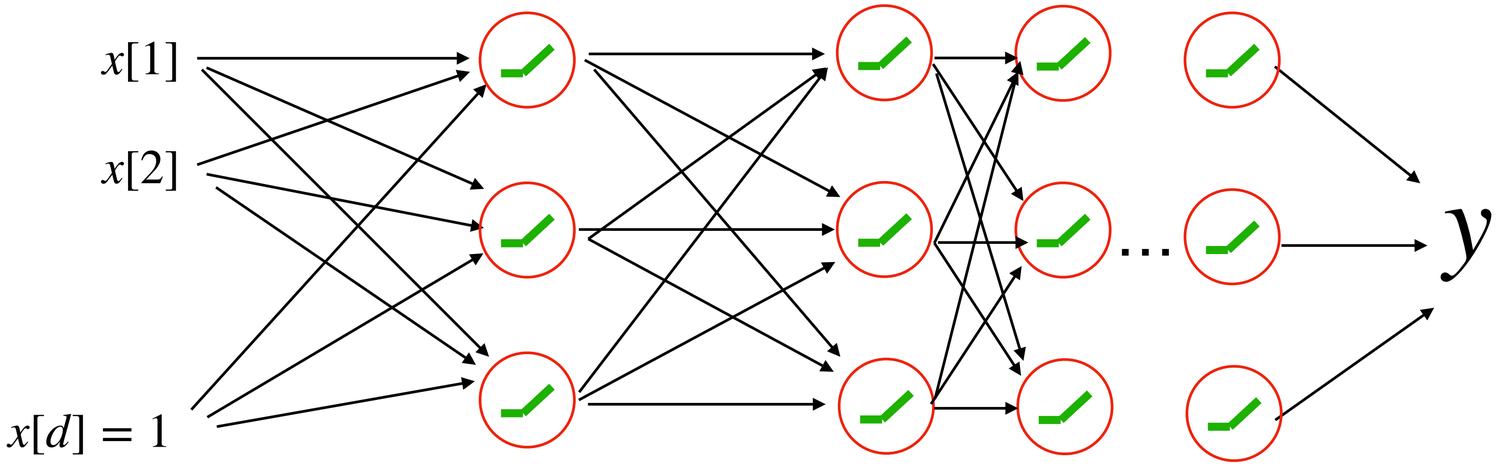
A multi-layer fully connected neural network



Define it by a forward pass:

$$z^{[1]} = x$$

A multi-layer fully connected neural network

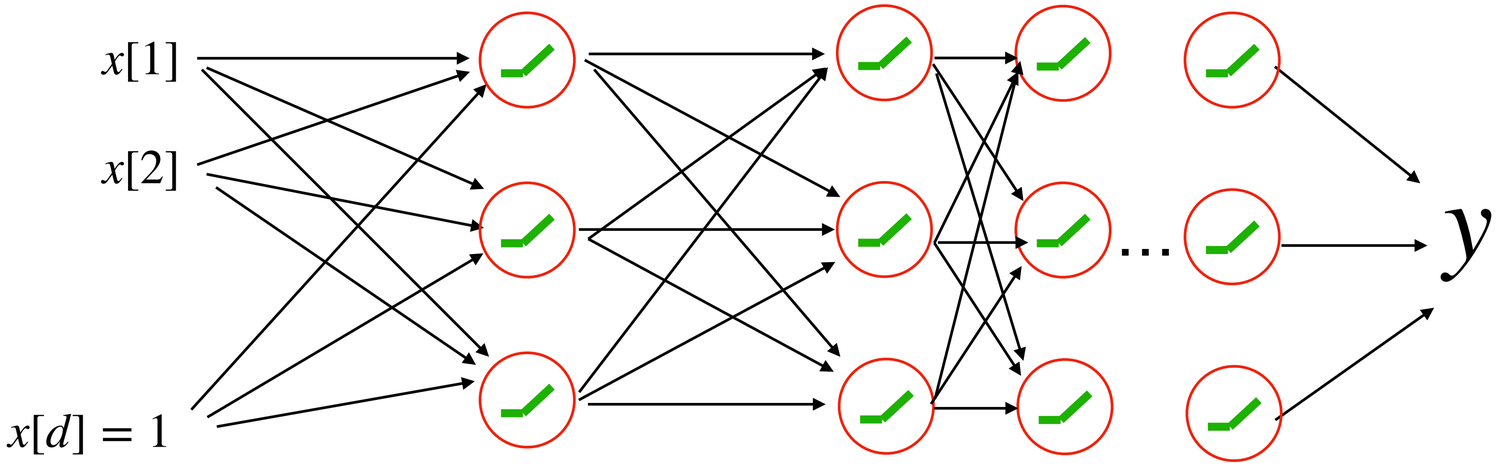


Define it by a forward pass:

$$z^{[1]} = x$$

For t = 1 to T-1:

A multi-layer fully connected neural network



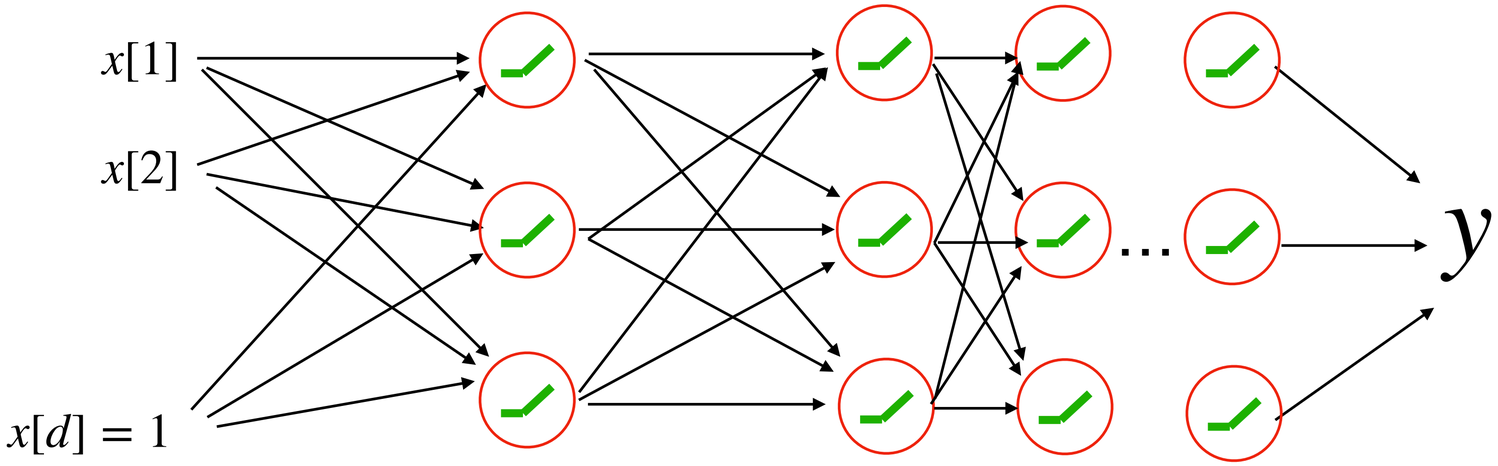
Define it by a forward pass:

$$z^{[1]} = x$$

For t = 1 to T-1:

$$z^{[t+1]} = \text{ReLU}\left(W^{[t]} z^{t}\right)$$

A multi-layer fully connected neural network



Define it by a forward pass:

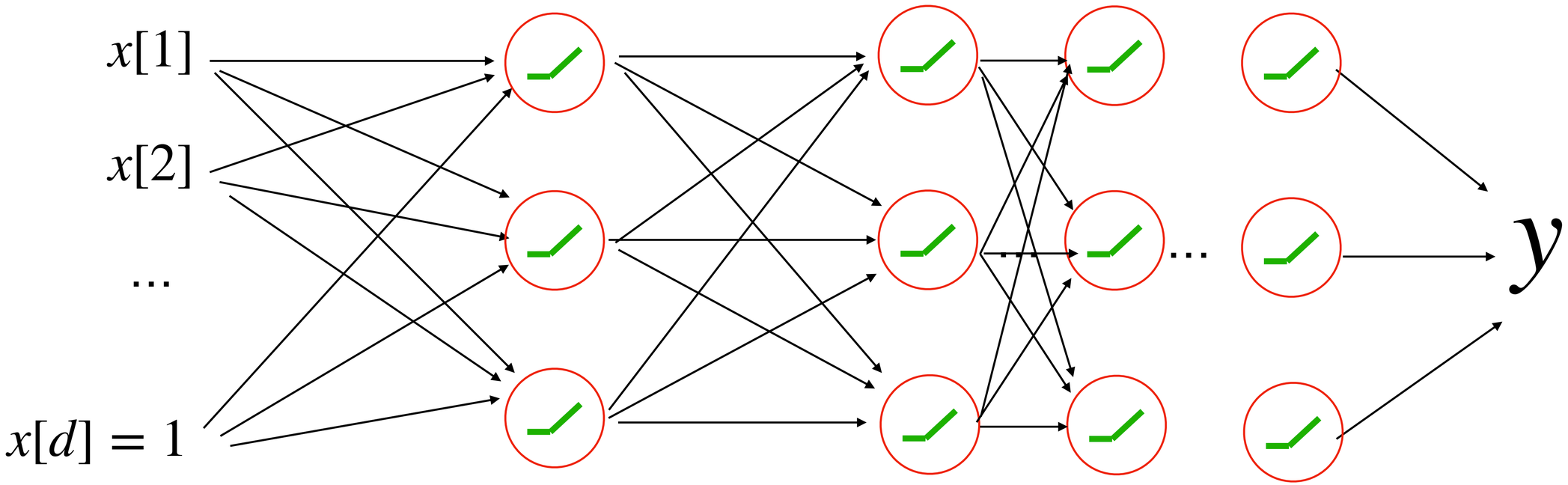
$$z^{[1]} = x$$

For t = 1 to T-1:

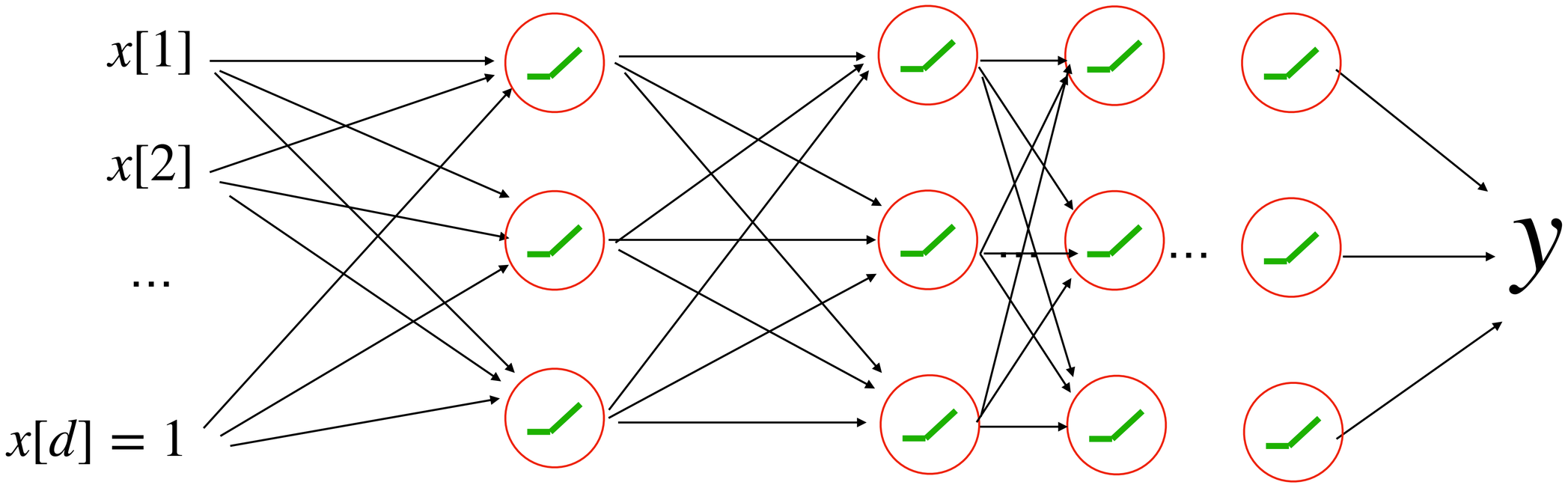
$$z^{[t+1]} = \text{ReLU}\left(W^{[t]} z^{t}\right)$$

$$y = \alpha^{\top} z^{[T]} + b$$

The benefits of going deep



The benefits of going deep



Allows us to represent complicated functions without making NN too wide

Outline of Today

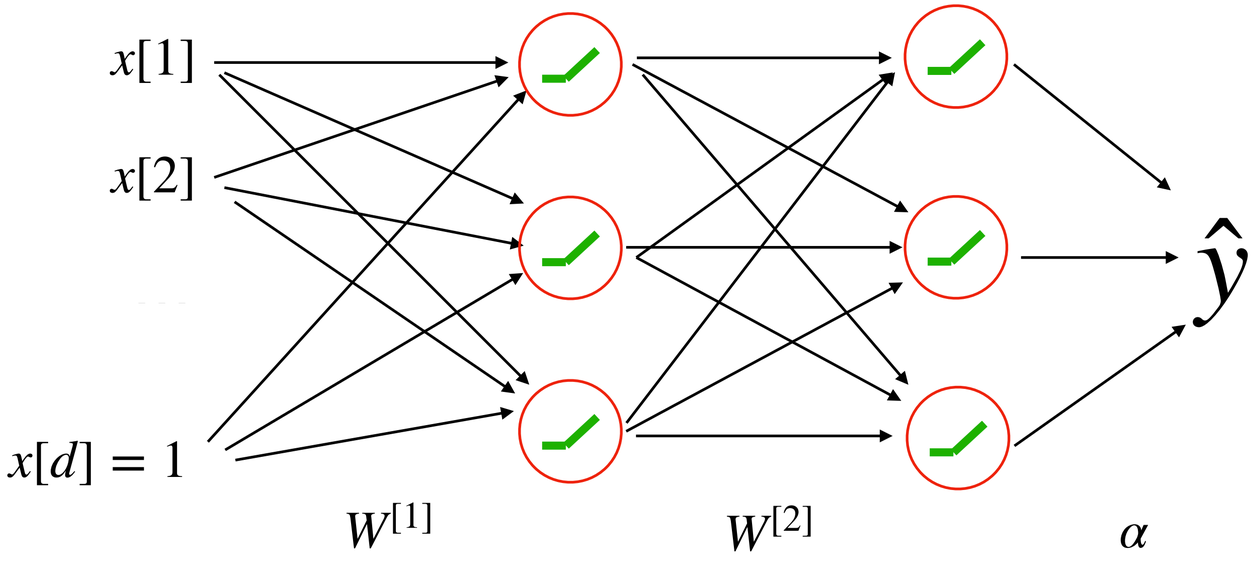
1. Analysis of Boosting

2. Multilayer feedforward Neural Network

3. Training a neural network

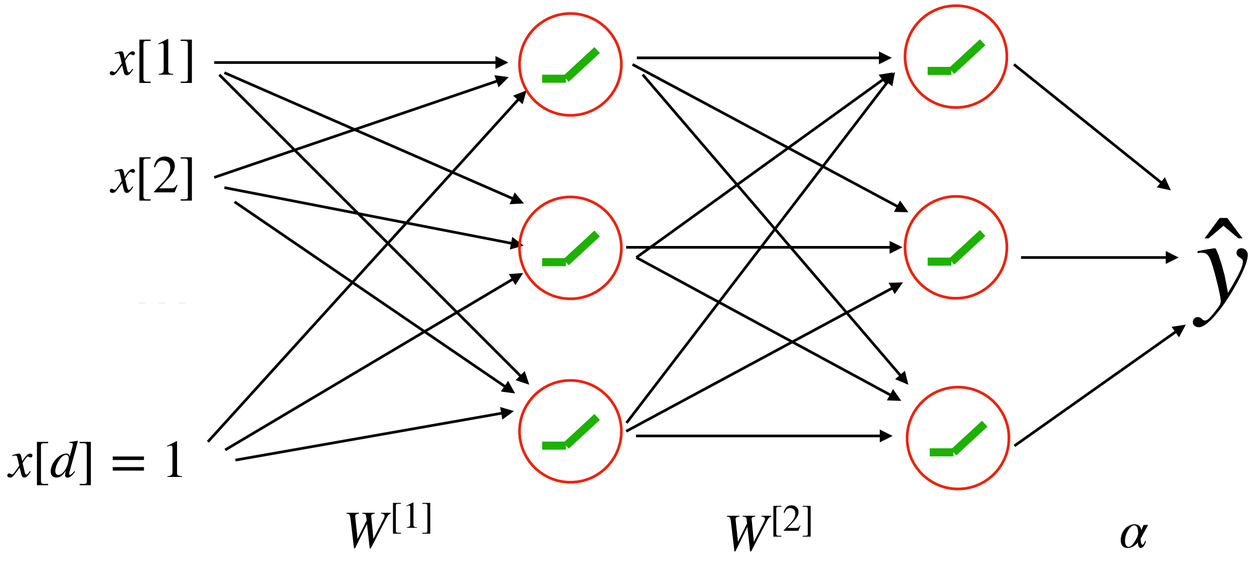
Training neural network via SGD

$$h(x) := \alpha^\top \text{ReLU}\left(W^{[2]}\text{ReLU}\left(W^{[1]}x\right)\right) + b$$



Training neural network via SGD

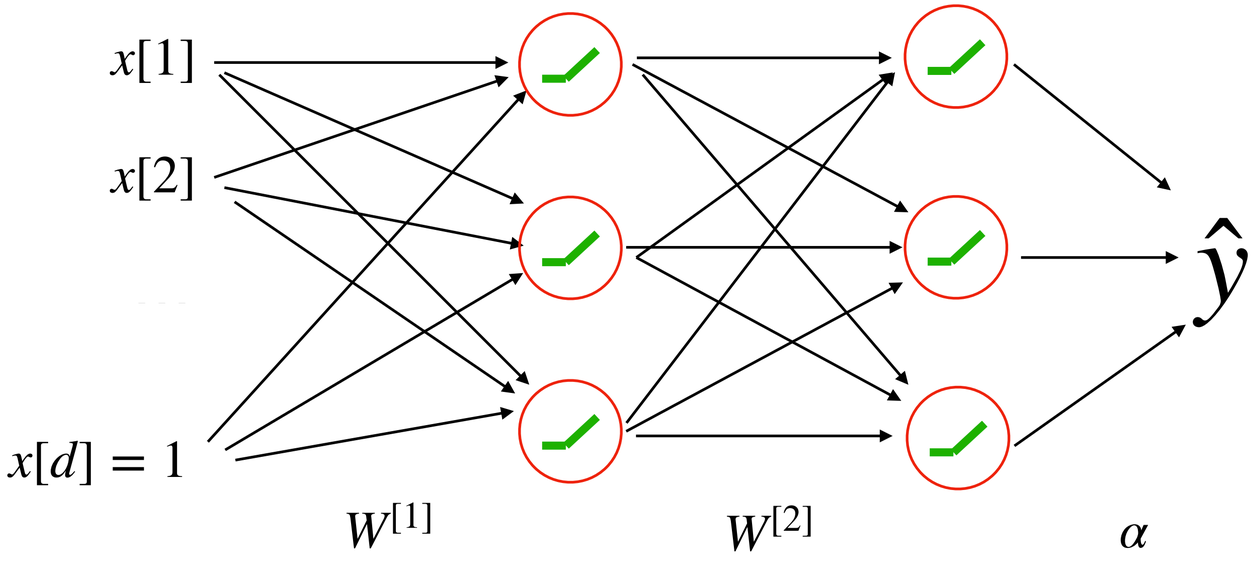
$$h(x) := \alpha^\top \text{ReLU}\left(W^{[2]}\text{ReLU}\left(W^{[1]}x\right)\right) + b$$



Let $\ell(h(x), y)$ be any differentiable loss function

Training neural network via SGD

$$h(x) := \alpha^\top \text{ReLU}\left(W^{[2]}\text{ReLU}\left(W^{[1]}x\right)\right) + b$$



Let $\ell(h(x), y)$ be any differentiable loss function

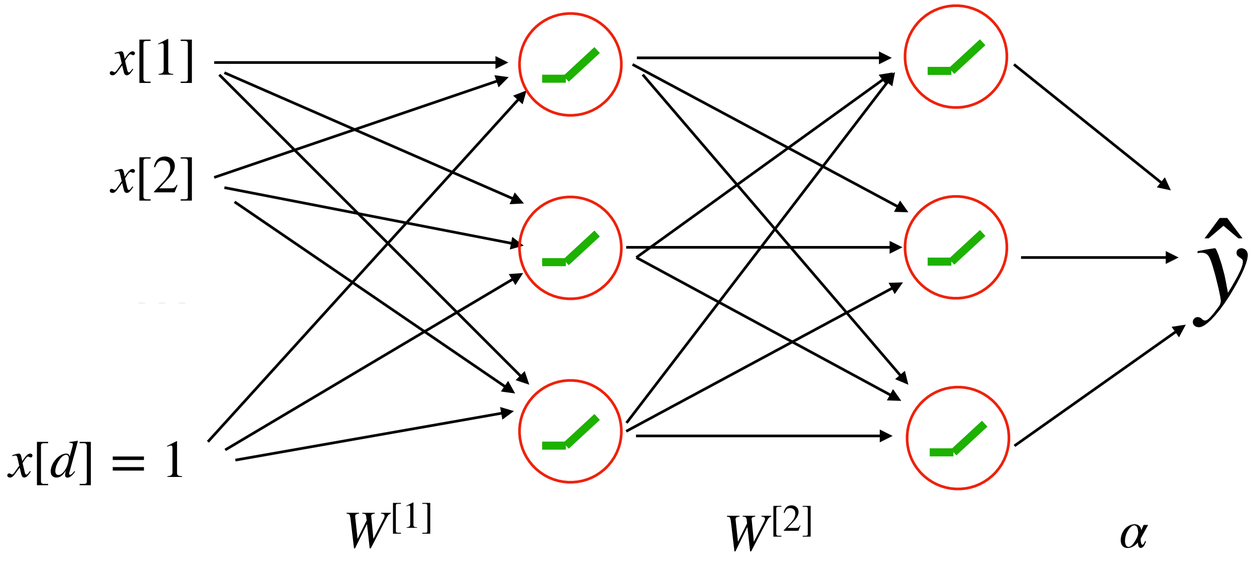
Compute gradients:

$$\frac{\partial \ell(h(x), y)}{\partial W^{[1]}} \qquad \frac{\partial \ell(h(x), y)}{\partial W^{[2]}}$$

$$\frac{\partial \ell(h(x), y)}{\alpha} \qquad \frac{\partial \ell(h(x), y)}{b}$$

Training neural network via SGD

$$h(x) := \alpha^\top \text{ReLU}\left(W^{[2]}\text{ReLU}\left(W^{[1]}x\right)\right) + b$$



Let $\ell(h(x), y)$ be any differentiable loss function

Compute gradients:

$$\frac{\partial \ell(h(x), y)}{\partial W^{[1]}} \qquad \frac{\partial \ell(h(x), y)}{\partial W^{[2]}}$$

$$\frac{\partial \ell(h(x), y)}{\alpha} \qquad \frac{\partial \ell(h(x), y)}{b}$$

(Next lecture: backpropagation for computing gradients)

Training neural network via SGD

Mini-batch Stochastic gradient descent

$\theta = [W^{[1]}, W^{[2]}, \alpha, b]$

For epoc $t = 1$ to T:

Training neural network via SGD

Mini-batch Stochastic gradient descent

$\theta = [W^{[1]}, W^{[2]}, \alpha, b]$

// go through dataset multiple times

For epoc $t = 1$ to T:

Training neural network via SGD

Mini-batch Stochastic gradient descent

$\theta = [W^{[1]}, W^{[2]}, \alpha, b]$

// go through dataset multiple times

For epoc $t = 1$ to T:

Randomly shuffle the data

Training neural network via SGD

Mini-batch Stochastic gradient descent

$\theta = [W^{[1]}, W^{[2]}, \alpha, b]$

For epoc $t = 1$ to T:

// go through dataset multiple times

Randomly shuffle the data

// important (unbiased estimate of the true gradient)

Training neural network via SGD

Mini-batch Stochastic gradient descent

$\theta = [W^{[1]}, W^{[2]}, \alpha, b]$

// go through dataset multiple times

For epoc $t = 1$ to T:

// important (unbiased estimate of the true gradient)

Randomly shuffle the data

Split the data into n/B many batches $\mathscr{D}_i$, each w/ size B

Training neural network via SGD

Mini-batch Stochastic gradient descent

$\theta = [W^{[1]}, W^{[2]}, \alpha, b]$

For epoc $t = 1$ to T:

// go through dataset multiple times

Randomly shuffle the data

// important (unbiased estimate of the true gradient)

Split the data into n/B many batches $\mathscr{D}_i$, each w/ size B

For j = 1 to n/B

Training neural network via SGD

Mini-batch Stochastic gradient descent

$\theta = [W^{[1]}, W^{[2]}, \alpha, b]$

For epoc $t = 1$ to T:

// go through dataset multiple times

Randomly shuffle the data

// important (unbiased estimate of the true gradient)

Split the data into n/B many batches $\mathscr{D}_i$, each w/ size B

For j = 1 to n/B

Mini-batch gradient $g = \sum_{x,y \in \mathscr{D}_i} \nabla_\theta \ell(h_\theta(x), y)/B$

Training neural network via SGD

Mini-batch Stochastic gradient descent

$\theta = [W^{[1]}, W^{[2]}, \alpha, b]$

For epoc $t = 1$ to T:

// go through dataset multiple times

Randomly shuffle the data

// important (unbiased estimate of the true gradient)

Split the data into n/B many batches $\mathscr{D}_i$, each w/ size B

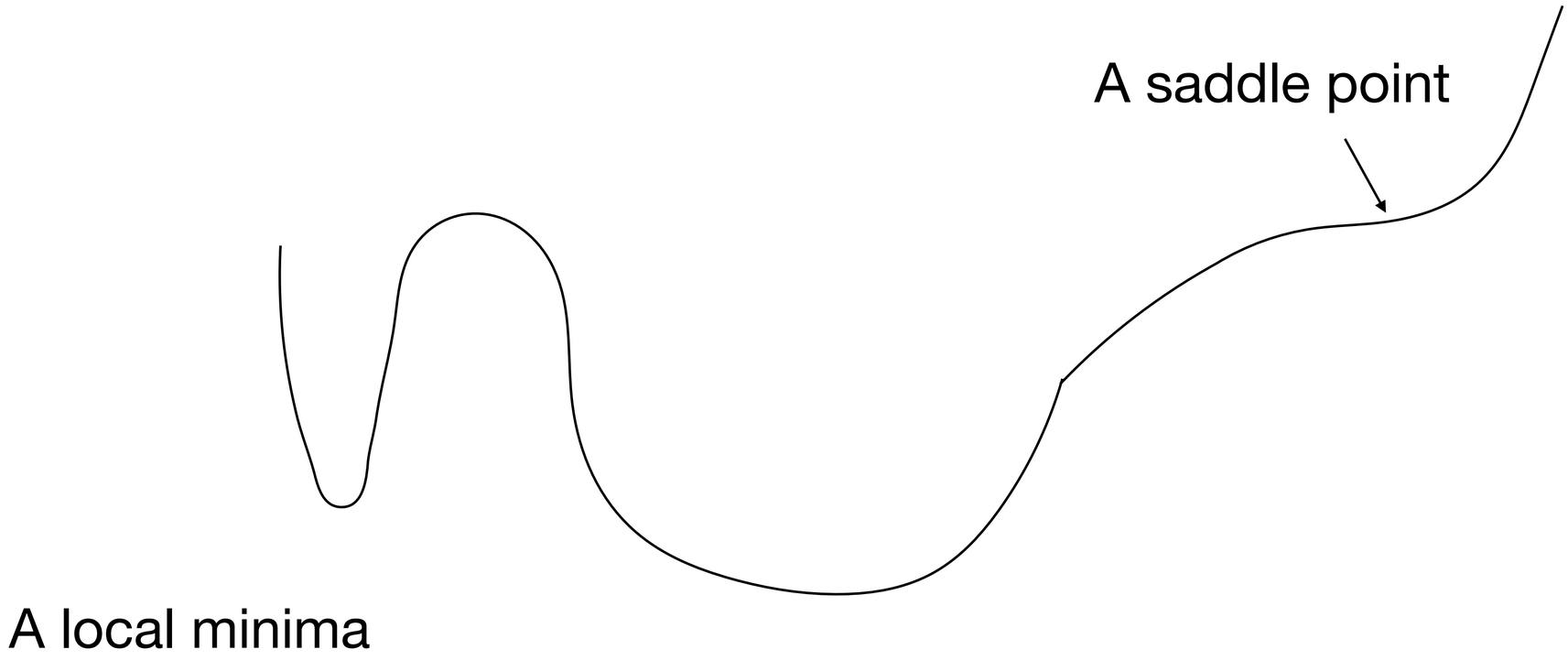
For j = 1 to n/B

Mini-batch gradient $g = \sum_{x,y \in \mathscr{D}_i} \nabla_\theta \ell(h_\theta(x), y)/B$

$\theta = \theta - \eta g$

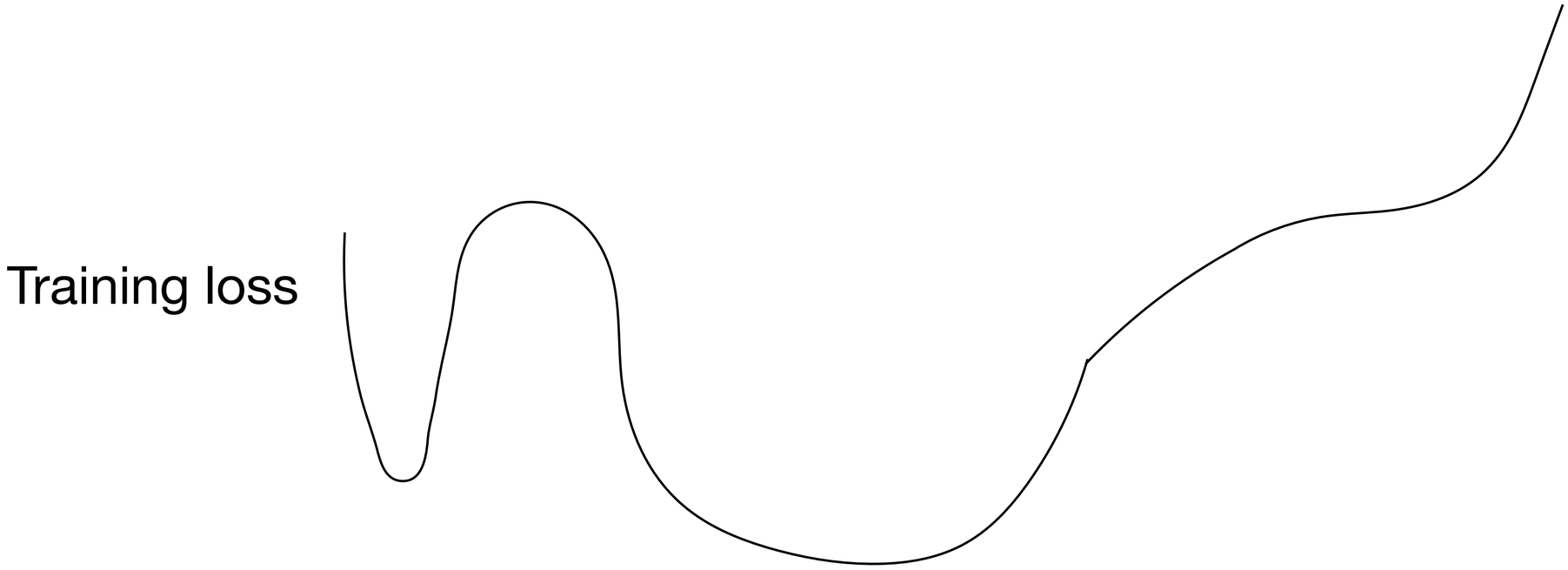
Training neural network via SGD

SGD helps avoiding local minima and saddle point



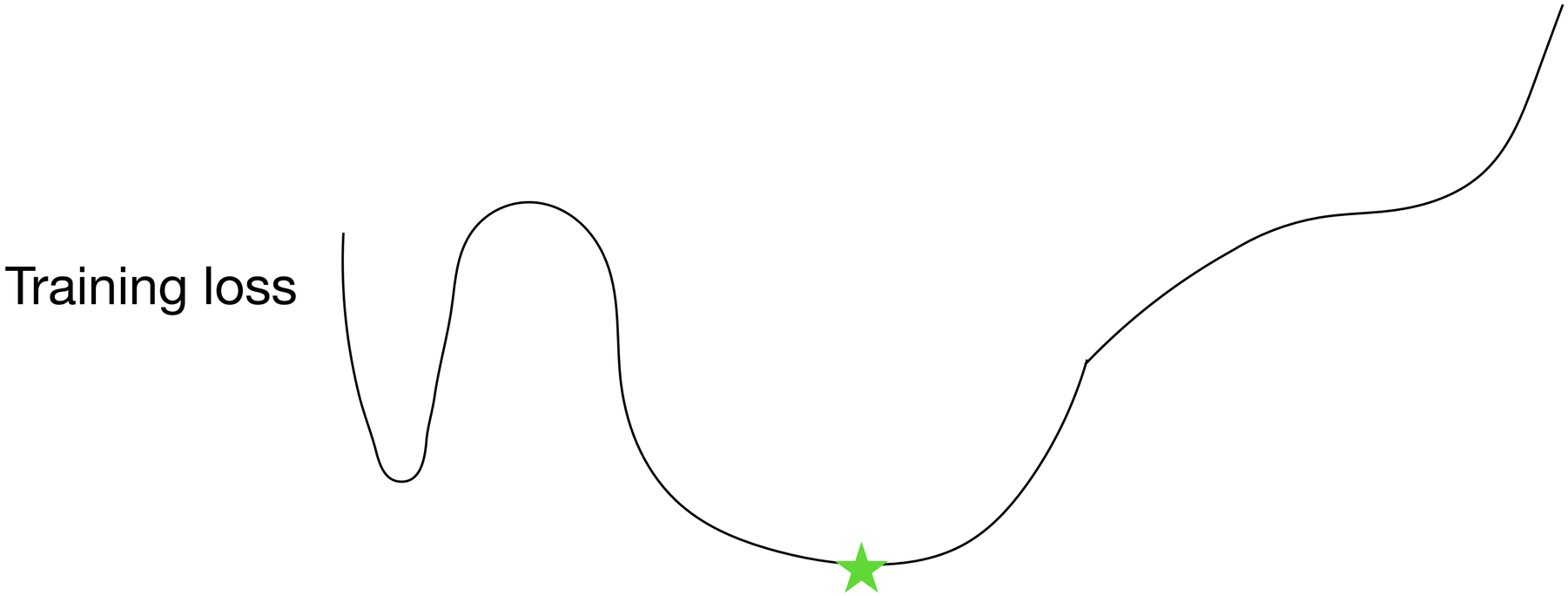
Training neural network via SGD

SGD tends to converge to a flat region



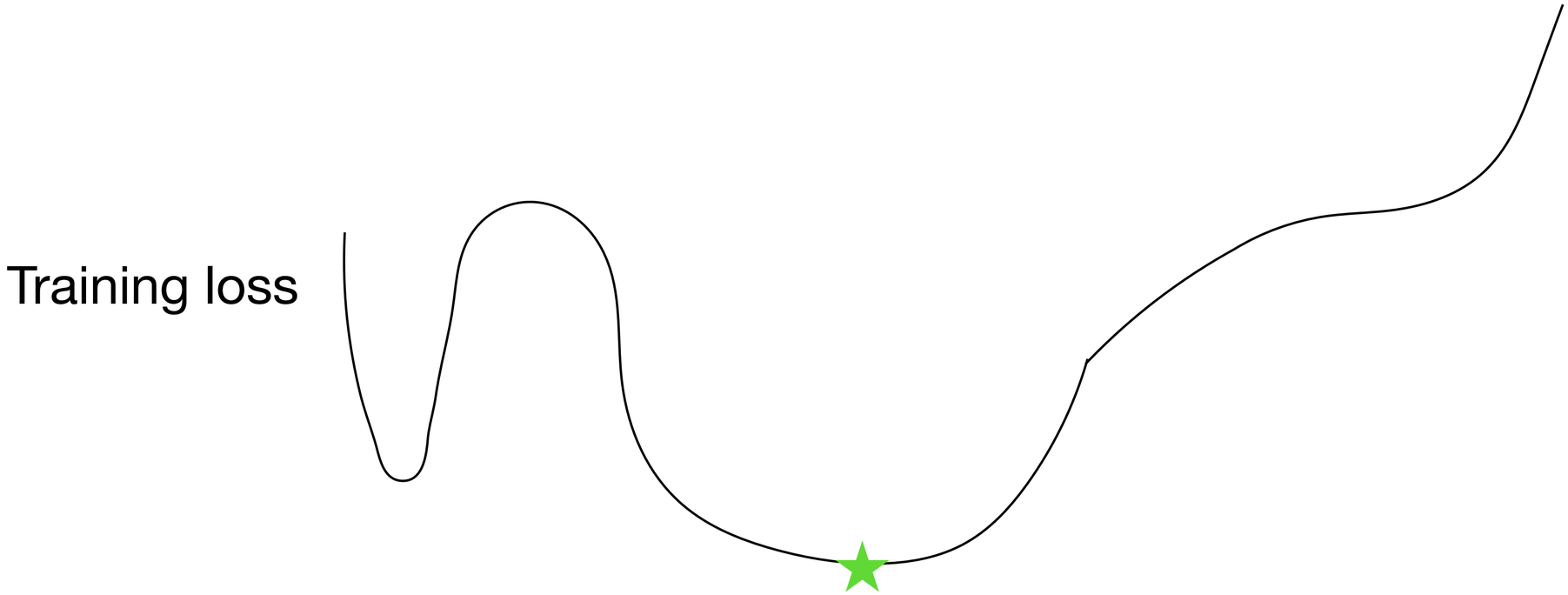
Training neural network via SGD

SGD tends to converge to a flat region



Training neural network via SGD

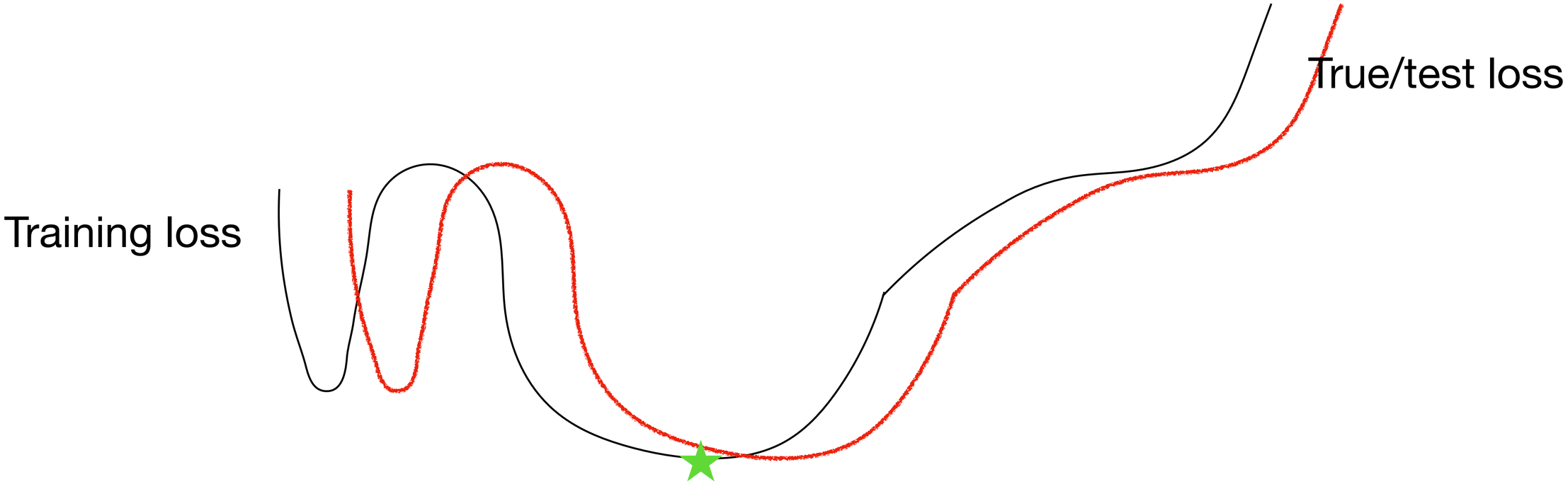
SGD tends to converge to a flat region



A flat local minima solution can help generalizes better to test data

Training neural network via SGD

SGD tends to converge to a flat region



A flat local minima solution can help generalizes better to test data

Summary for today

1. Neural network is universal function approximation

2. SGD is important for training neural networks

Next lecture: backpropagation